

Physics Concepts and Fundamentals

Generic introduction to fundamental concepts in physics and
mathematics

September 18, 2026

Contents

1	Generic Mathematical Tools	3
1.1	Inner Product	3
1.2	Cartesian Product	3
1.3	Tensor and Outer Product	3
1.4	Lagrangian Multipliers	7
1.5	Gradient and Jacobian	8
1.6	Legendre Transform	9
1.7	Differentials	10
1.8	Integrals	10
1.9	Differential Equations	10
2	Newtonian Mechanics	13
2.1	Hooke's Law	13
3	Lagrangian Mechanics	15
3.1	Mass Matrix	15
3.2	Lagrange's Equations	16
4	Hamiltonian Mechanics	17
4.1	Phase Space	17
5	Statistical Mechanics	18
5.1	Canonical Ensemble	18
5.2	Grand Canonical Ensemble	19
6	Fourier Analysis	19
7	Information Theory	21
8	Game Theory	21
8.1	Shapley Value	21

9	Network Theory	22
9.1	Types of Matrixes	22
9.2	Types of Networks	24
9.3	Centralities	25
9.4	Diffusion on Graphs (analytical)	29
9.5	Experimental Diffusion	30
9.6	Path Finding	30
9.7	Link Prediction	32
10	Markov Processes	33
11	Time Series	33
11.1	Theoretical Description	34
11.2	Technical Difficulties	35
11.3	Algorithms	35
12	Likelihood Function	40
12.1	Probability Functions	40
12.2	Maximum Likelihood Estimation	41
13	P-Value	41
14	Monte Carlo Simulation	42
15	Machine Learning Techniques	42
15.1	K-Mean Clustering	43
15.2	Principal Component Analysis (PCA)	43
15.3	Gradient Descent	44
15.4	Gradient Boosting	45
15.5	Simple Neural Networks	46
15.6	Reinforcement Learning	50
15.7	Transformers	54
15.8	Model Performances Evaluation	55
16	Relativity	59
17	Quantum Mechanics	59
17.1	Hilber Space	59
17.2	Nomenclature	60
17.3	State Vectors, Basis and Descriptions	61
17.4	Define Observable Operators	63
17.4.1	Commutator	64
17.4.2	Generating Observables	64
17.5	Schrödinger Equation	65

1 Generic Mathematical Tools

1.1 Inner Product

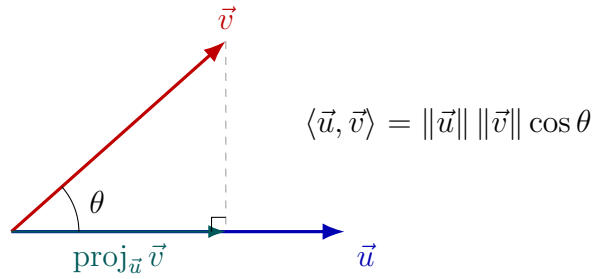
Let V be a space vector and F be the field on which the vector space is defined. The inner product – or dot/scalar product – is a bilinear transformation $\langle \cdot, \cdot \rangle$ such that:

$$\langle \cdot, \cdot \rangle : V \times V \rightarrow F$$

$$\vec{v}, \vec{u} \mapsto \langle \vec{v}, \vec{u} \rangle = \sum_{i=1}^N v_i u_i$$

In vector form we can write the inner product as:

$$u^\top v = \begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} = \sum_{i=1}^N v_i u_i$$



1.2 Cartesian Product

The Cartesian product of two vector spaces $V \times U$ is defined simply as the ordered pairs of all the vectors of V and U :

$$V \times U = \{(v, u), v \in V, u \in U\}$$

and it is usually used in function definition when describing from which space a function should take its arguments. The components remain separated and do not mix ($\dim V \times U = \dim V + \dim U$). It usually gets confused with the tensor product, which is a completely different concept which takes vector from different spaces and mix them bilinearly so that we can expand one space degrees of freedom with another one ($\dim V \otimes U = \dim V \cdot \dim U$).

1.3 Tensor and Outer Product

The tensor product is an operator $\otimes$ that combines two vector spaces V and U into a bigger one $V \otimes U$ that contains all bilinear information between them. This means that given $v \in V$ and $u \in U$:

- $(v_1 + v_2) \otimes u = v_1 \otimes u + v_2 \otimes u$

- $v \otimes (u_1 + u_2) = v \otimes u_1 + v \otimes u_2$
- $(\lambda v) \otimes u = v \otimes (\lambda u)$

It's crucial to notice that, although $(v \otimes u) \in (V \otimes U)$, not all elements in the space $V \otimes U$ can be written as $u \otimes v$. In fact, given $w \in (V \otimes U)$ we have that $w = \sum_{i,j} (c_{i,j} (e_i \otimes f_j))$, with $\{e_i\}$ being a base for V and $\{f_j\}$ a base for U . We can actually show this using the bilinearity, assuming $V = U = \mathbb{R}^2$:

- $w \equiv e_1 \otimes f_1 + e_2 \otimes f_2$
- assume that exists $w = v \otimes u$ with $v = ae_1 + be_2$ and $u = cf_1 + df_2$
- since it is bilinear $v \otimes u = ac(e_1 \otimes f_1) + ad(e_1 \otimes f_2) + bc(e_2 \otimes f_1) + bd(e_2 \otimes f_2)$
- by the definition we gave of w we have $ac = 1, ad = 0, bc = 0, bd = 1$
- we got to an absurd with only a combination of the two vectors!

This means that the element w can be obtained only if we take the linear combination of the tensor product of the all the pairings of two spaces basis vectors. We can prove that the basis for $V \otimes U$ is $\{e_i \otimes f_j : i = 1, \dots, N, j = 1, \dots, M\}$, and so that's why any element of that space can be written as:

$$w = \sum_i \sum_j (c_{i,j} (e_i \otimes f_j)) .$$

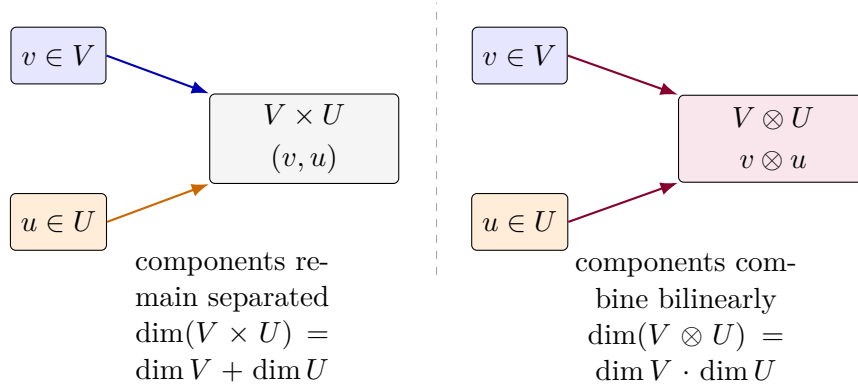
This introduces the concept of simple tensor – or rank-1 tensor since it is a matrix with only one independent vector – which is any vector in $V \otimes U$ that can be written using coefficients $c_{i,j} = a_i b_j$ with a_i coming from the basis of V ($v = \sum_i a_i e_i$) and b_j from the basis of U ($u = \sum_j b_j f_j$).

The outer product is simply a subset of the tensor product once a basis is fixed. In particular we define outer product of two vectors v, u :

$$v \otimes u = v u^\top = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_m \end{pmatrix} \begin{pmatrix} u_1 & u_2 & \cdots & u_n \end{pmatrix} = \begin{pmatrix} v_1 u_1 & v_1 u_2 & \cdots & v_1 u_n \\ v_2 u_1 & v_2 u_2 & \cdots & v_2 u_n \\ \vdots & \vdots & \ddots & \vdots \\ v_m u_1 & v_m u_2 & \cdots & v_m u_n \end{pmatrix}$$

where V and U can be of different dimensions, and the resulting matrix will always be of rank-1 (one single independent vector). We used the same nomenclature of the tensor product because this it's exactly the same thing, but considering only one basis vector of $V \otimes U$ – instead of $(v_1 \otimes u_1 + v_2 \otimes u_1 + \dots)$.

One cool application of tensor products is in the entanglement in quantum mechanics (as we will see later). A quantum particle A lives in a space V , another B in a space U . Their combined system can be obtained as the tensor product of the two $V \otimes U$, since we want to consider all possible combinations of their states. States in $V \otimes U$ can be separable or entangled. Separable states are simple tensors – rank-1 tensors, so the outer product – while entangled states are responsible for correlations between the particles that can only be explained by higher rank tensors.



Tensor Index Formalism

Tensor index formalism is a compact notation that helps us describe tensors and their operations without writing every component explicitly. The main idea is simple: each index tells us which component of a vector, matrix, or higher order tensor we are considering.

For example, in a vector $v \in \mathbb{R}^n$ with components:

$$v_i, \quad i = 1, \dots, n.$$

the index i selects indeed only the i -th element of that vector.

Considering a matrix $A \in \mathbb{R}^{m \times n}$ has two indices:

$$A_{ij},$$

where i identifies the row and j identifies the column. Therefore, the entry in the i -th row and j -th column is A_{ij} . More generally, a tensor of order r has r indices, for example a third-order tensor has three indices: T_{ijk} .

The tensor product between two vectors $v \in V$ and $u \in U$ can be written in components as:

$$(v \otimes u)_{ij} = v_i u_j.$$

This is exactly the outer product seen before. The first index belongs to v , while the second index belongs to u . In particular, if $v \in \mathbb{R}^m$ and $u \in \mathbb{R}^n$, then:

$$(v \otimes u)_{ij} = v_i u_j,$$

which corresponds to the matrix:

$$v \otimes u = \begin{pmatrix} v_1 u_1 & v_1 u_2 & \cdots & v_1 u_n \\ v_2 u_1 & v_2 u_2 & \cdots & v_2 u_n \\ \vdots & \vdots & \ddots & \vdots \\ v_m u_1 & v_m u_2 & \cdots & v_m u_n \end{pmatrix}.$$

A very useful convention is the Einstein summation convention. Whenever an index appears twice in the same term, we automatically sum over all its possible values. For instance:

$$w_i = A_{ij} v_j \rightarrow \sum_{j=1}^n A_{ij} v_j.$$

This is simply the i -th component of the matrix-vector multiplication:

$$(Av)_i = A_{ij} v_j.$$

Any index appearing once is called *free index*, while an index appearing twice is a *summed index*. The inner product $v, u \in \mathbb{R}^n$, which only has a single summed index, becomes:

$$v_i u_i \rightarrow \sum_{i=1}^n v_i u_i.$$

In tensor notation, the position of an index matters. An index written below, as in v_i , is called a *lower index*, while an index written above, as in v^i , is called an *upper index*.

Upper indices are typically used for the vector components. Lower indices are typically used for covectors components, which are linear maps that take a vector as input and return a scalar. For instance, the expressions

$$\omega_i v^i$$

means that a covector ω_i acts on a vector v^i , and the repeated index implies summation. This could also be written with a more usual structure as:

$$\omega_i v^i = \omega(\vec{v}) ,$$

but it lacks the explicit representation that the i -th component of the covector acts on the i -th component of the vector. In Euclidean space, once the metric is used to identify vectors and covectors, this operation can be written as the usual scalar product: if $\vec{v} \in \mathbb{R}^N$ and ω is a covector, with associated vector $\vec{\omega} \in \mathbb{R}^N$, then

$$\omega_i v^i = \omega(\vec{v}) = \vec{\omega}^\top \vec{v}.$$

Usually, if we fix a base, the covectors are "row vectors", while the vectors are the "column vectors", which is useful to keep in mind even tho it's not that general.

The *metric tensor*, usually denoted by g_{ij} , allows us to pass from vectors to covectors and viceversa. More precisely, if v^j are the components of a vector, then the corresponding covector with lowered index is defined by

$$v_i = g_{ij} v^j.$$

This operation is called "lowering an index".

Conversely, if ω_j are the components of a covector, then we can recover a quantity with an upper index by using the inverse metric g^{ij} :

$$\omega^i = g^{ij} \omega_j.$$

This operation is called "raising an index".

Therefore, raising and lowering indices are the procedures that allow us to move between vectors and covectors through the metric. They do not simply change the position of the index in a formal way, but they modify the components according to the geometry encoded by the metric. For example, in $\mathbb{R}^2$ with the standard Euclidean metric,

$$g_{ij} = \delta_{ij} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix},$$

consider the vector with components

$$\vec{v} = \begin{pmatrix} v^1 = 2 \\ v^2 = 3 \end{pmatrix} .$$

Lowering the index gives

$$v_i = g_{ij}v^j,$$

hence

$$\begin{aligned} v_1 &= 1 \cdot 2 + 0 \cdot 3 = 2 \\ v_2 &= 0 \cdot 2 + 1 \cdot 3 = 3. \end{aligned}$$

So in this case the lowered components are the same:

$$(v_1, v_2) = (2, 3).$$

Similarly, if a covector has components

$$\omega_1 = 4, \quad \omega_2 = -1,$$

then raising the index gives

$$\omega^i = g^{ij}\omega_j,$$

and since $g^{ij} = \delta^{ij}$, we obtain

$$(\omega^1, \omega^2) = (4, -1).$$

This means that, in standard Euclidean coordinates, raising and lowering indices does not change the numerical values of the components, although the meaning of the object changes: v^i is viewed as a vector, while v_i is viewed as the corresponding covector. However, in spaces where we do not have the identity matrix as metric, the values will change. Also, it's worth noting that we can have infinite covectors for any vector v^i – and viceversa – but v_i it's special because:

- It is directly determined by the metric g_{ij} .
- It is linear in v .
- It gives the usual "inner product" between any vector u with v , which is the most crucial aspect. If we want to evaluate the usual inner product between v^i and any other vector u^i , we simply consider $v_i = g_{ij}v^j$.

1.4 Lagrangian Multipliers

Lagrangian multipliers are a method for finding the maximum or minimum of a function f under some constraints. Let $f(\vec{x})$ be our function, and let $g(\vec{x}) = c$ be our constraint – meaning that the components x_i must satisfy that condition. Normally we would just require $\nabla f = 0$ since we are free to move linearly wherever we want, while in this case we can only move on an $(N-1)$ -dimensional surface described by $g(\vec{x}) = c$.

Since we are on an hyper surface, the tangent hyper plane to the curve at point $\vec{x}_0$ must be the same for both g and f up to a multiplicative constants:

$$\nabla f(\vec{x}_0) = \lambda \nabla g(\vec{x}_0),$$

where λ is just a multiplicative constant for now.

Also, since both f and g are defined on the same points, we can actually write a new function

$$L(\vec{x}, \lambda) = f(\vec{x}) - \lambda(g(\vec{x}) - c),$$

where λ is called Lagrangian multiplier. We have as many Lagrangian multipliers as constraints in our system.

In this way we can transform our constrained problem with N variables, into an unconstrained one with $N + 1$ variables (one more per lagrange multiplier we have, i.e. constrain). Without this, we would need to explicitly parametrize the surface and evaluate the gradient. In fact, we can see that this equation actually recovers both the gradient condition on f and g – same tangent plane:

$$\nabla_{\vec{x}} L(\vec{x}, \lambda) = 0 \rightarrow \nabla f(\vec{x}_0) = \lambda \nabla g(\vec{x}_0)$$

and the hyper surface constrain:

$$\nabla_{\lambda} L(\vec{x}, \lambda) = 0 \rightarrow g(\vec{x}) = c .$$

Solving these systems will give us candidate for maximum, minimum or saddle points.

1.5 Gradient and Jacobian

Let $f : \mathbb{R}^N \rightarrow \mathbb{R}$, and let $\nabla f : \mathbb{R}^N \rightarrow \mathbb{R}^N$ be the gradient of f such that:

$$\nabla f(\vec{x}) = \begin{bmatrix} \frac{\partial f(\vec{x})}{\partial x_1} \\ \frac{\partial f(\vec{x})}{\partial x_2} \\ \dots \\ \frac{\partial f(\vec{x})}{\partial x_N} \end{bmatrix} .$$

The gradient represents the rate of change of the function f while varying a generic variable. Since a function is defined from a vector space A to a vector space B , the gradient will be a column vector living in space A . Also, the gradient vector will be orthogonal to the function domain in A , and its length will determine the increase rate of the function in B . The reason why the gradient is orthogonal to the level curve is the following: Suppose to have a function

$$\begin{aligned} f : \mathbb{R}^N &\rightarrow \mathbb{R} \\ \vec{x} &\mapsto f(\vec{x}) = c \end{aligned}$$

and $\vec{r}(q)$ is a parametrization of the level curve – i.e. a way to describe it using parameters, like $y = x^2 \rightarrow x = q, y = q^2 \rightarrow \vec{r}(q) = (q, q^2)$. Since c is a constant, using the chain rule we have:

$$\frac{d f(\vec{r}(q))}{dq} = \nabla f(\vec{r}(q)) \cdot \frac{d \vec{r}(q)}{dq} = 0 ,$$

and since $\frac{d \vec{r}(q)}{dq}$ is tangent to the level curve, the gradient must be orthogonal due to the scalar product being zero.

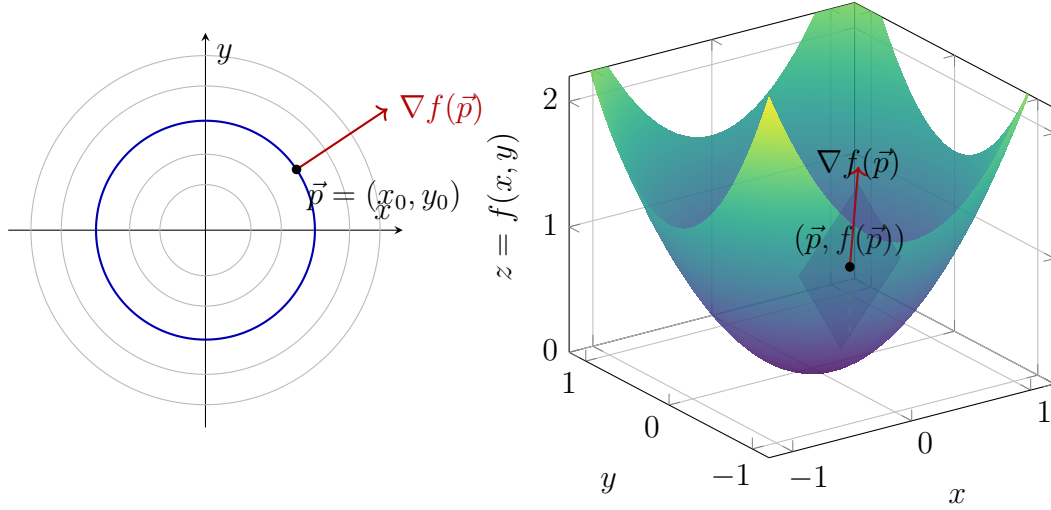


Figure 1: *Left: the gradient $\nabla f(\vec{p})$ in the domain, orthogonal to the level curve of f through $\vec{p}$. Right: the same point lifted onto the surface $z = f(x, y)$, with $\nabla f(\vec{p})$ shown as tangent to the surface.*

The Jacobian matrix instead is the same concept but for a function defined as $f : \mathbb{R}^N \rightarrow \mathbb{R}^M$:

$$J_f = \begin{bmatrix} \frac{\partial \vec{f}}{\partial x_1} & \frac{\partial \vec{f}}{\partial x_2} & \cdots & \frac{\partial \vec{f}}{\partial x_N} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_N} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_M}{\partial x_1} & \frac{\partial f_M}{\partial x_2} & \cdots & \frac{\partial f_M}{\partial x_N} \end{bmatrix} = \begin{bmatrix} \nabla f_1(\vec{x}) \\ \nabla f_2(\vec{x}) \\ \vdots \\ \nabla f_N(\vec{x}) \end{bmatrix}$$

The reason why the Jacobian has the gradients as rows and not as columns is just a convention, for instance to match first order Taylor expansion:

$$\vec{f}(\vec{x} + \vec{h}) \simeq \vec{f}(\vec{x}) + J\vec{f}(\vec{x})(\vec{h})$$

instead of using the transpose.

1.6 Legendre Transform

Method to rewrite a convex function as a different one replacing one variable with the derivative of the original function:

$$L_T(f(\vec{x}))(\vec{p}) = \sup_{\vec{x} \in \mathbb{R}^N} [\langle \vec{p}, \vec{x} \rangle - f(\vec{x})] , \quad (1)$$

Where it emerges that the sup is obtained exactly at a certain $\vec{x}^* = \nabla f(\vec{x}^*) \equiv \vec{p}$.

This transform is crucial in Hamiltonian mechanics to switch between a second order differential equation, to a system of first order differential equations (Hamilton equations).

1.7 Differentials

Let $f(\vec{x})$ be a multivariate function. The differential of f can be defined as the overall infinitesimal displacement of f in any direction:

$$df(\vec{x}) = \frac{\partial f}{\partial x_1} dx_1 + \frac{\partial f}{\partial x_2} dx_2 + \cdots + \frac{\partial f}{\partial x_n} dx_n . \quad (2)$$

The differential is a linear operator in the dual space of the tangent plane of f in $\vec{x}$ on the manifold M where the function lives:

$$df_{\vec{x}} : T_{\vec{x}}M \rightarrow \mathbb{R} , \quad df_{\vec{x}} \in T_{\vec{x}}^*M$$
$$\vec{v} \mapsto df_{\vec{x}}(\vec{v}) = \sum_{i=1}^n \left. \frac{\partial f}{\partial x_i} \right|_{\vec{x}} v_i$$

1.8 Integrals

Generalization of an infinite sum of a function over an interval divided in infinitesimal steps equally spaced. We start by considering the finite steps Δx ..

Complex integral ...

1.9 Differential Equations

Differential equations are used to write mathematical relationships between a quantity and its derivatives. In other words, they are used to model how a quantity is affected when it changes according to another parameter, that could be time, position etc..

The most straightforward example is the population dynamics:

$$\frac{dN(t)}{dt} = -\alpha N(t) + \beta N(t) = (\beta - \alpha)N(t)$$
$$N(0) = N_0 ,$$

where α is the death rate parameter and β is the birth rate parameter. It says: the population $N(t)$ varies according to the current population $N(t)$ weighted by the birth-death rate parameters.

The differential can be:

- **Linear** : The variable and its derivatives appear at first order power

$$y'' - y' = 4$$

- **Homogeneous** : There are no constant terms

$$y'' - y'^2 = 0$$

Also, the order of a differential equation is the maximum derivative present. This is an example of a 3-rd order, *non-linear* and *non-homogeneous* differential equation:

$$\frac{d^3 y}{dt^3} + 3 \frac{dy^3}{dt} - 6 = t^2 .$$

or also

$$\frac{d^3y}{dt^3} + 3 \left(\frac{dy}{dt} \right)^3 - 6 = t^2 .$$

An ODE – Ordinary Differential Equation – is a differential equations that contains a *single* independent variable, for instance time. $y'' + 3y - 5t = 3$ is an ODE, $y'' + 3y^2 = 0$ is not.

A PDE – Partial Differential Equation – is instead an equation that contains more independent variables, for instance time and position, like this one:

$$\frac{\partial^3 y(x, t)}{\partial t^3} - 2 \frac{\partial y^2(x, t)}{\partial x} = tx^2 + 1 .$$

General Solution Techniques

The appropriate solution technique depends on the structure of the differential equation. Some common methods are:

- **Separation of variables:** used when the equation can be written as

$$\frac{dy}{dt} = f(t)g(y) .$$

The variables are separated and both sides are integrated:

$$\int \frac{1}{g(y)} dy = \int f(t) dt .$$

- **Integrating factor:** used for first-order linear ODEs of the form

$$y' + p(t)y = q(t) .$$

The equation is multiplied by the integrating factor

$$\mu(t) = e^{\int p(t) dt} .$$

- **Exponential ANSATZ and characteristic equation:** used for linear homogeneous ODEs with constant coefficients. For example,

$$ay'' + by' + cy = 0 .$$

We assume a trial solution – ANSATZ – of the form

$$y(t) = e^{rt} ,$$

because its derivatives are proportional to the function itself:

$$y'(t) = re^{rt}, \quad y''(t) = r^2 e^{rt} .$$

Substituting the ansatz into the ODE gives

$$e^{rt} (ar^2 + br + c) = 0 .$$

Since $e^{rt} \neq 0$, the roots are found from the characteristic equation

$$ar^2 + br + c = 0 .$$

- **Substitution:** a change of variables can transform a non-linear or complicated equation into a simpler one.
- **Laplace transform:** converts a differential equation into an algebraic equation and is particularly useful for initial-value problems.
- **Series expansion:** approximates the solution as a power series when a closed-form solution cannot be found.

For PDEs, common analytical techniques include separation of variables, Fourier transforms, and the method of characteristic equations. In real-life application, we usually integrate the equations numerically – ex. toxin diffusion on brain, weather prediction.

Numerical Integration

Many differential equations cannot be solved analytically. In these cases, the solution is approximated numerically at discrete points:

$$t_n = t_0 + nh ,$$

where h is the integration step.

For an initial-value problem

$$\begin{aligned} y'(t) &= f(t, y(t)) , \\ y(t_0) &= y_0 , \end{aligned}$$

the simplest numerical method is the **explicit Euler method**:

$$y_{n+1} = y_n + hf(t_n, y_n) .$$

Smaller values of h generally improve the accuracy but require more computations. More accurate and commonly used techniques include:

- **Improved Euler or Heun method.** Fast, but not that precise for long predictions.
- **Runge–Kutta methods**, especially the fourth-order method. Very precise, but computational expensive.
- **Implicit methods**, which are useful for stiff equations.
- **Finite-difference, finite-volume, and finite-element methods** for PDEs.

Numerical methods provide an approximation rather than an exact formula, so their accuracy and stability must always be considered. It's important also to remember that the longer the prediction, the higher the error, and thus the quantities are not conserved. Symplectic integrators preserves specific quantities – like energy in hamiltonian dynamics – but are generally less precise point-wise.

2 Newtonian Mechanics

Solve the dynamics of a system by modeling it to a vectorial problem and applying the principles in order to get a second order differential equation.

- $\vec{r}(t)$: position $\longrightarrow \frac{d}{dt}(\vec{r}(t)) = \vec{v}(t) \equiv \dot{\vec{x}}(t)$: velocity
- $\vec{v}(t)$: velocity $\longrightarrow \frac{d}{dt}(\vec{v}(t)) = \vec{a}(t) \equiv \dot{\vec{v}}(t) = \ddot{\vec{x}}(t)$: acceleration

The second principle of Newtonian mechanics is:

$$\vec{F}(t) = m\vec{a}(t) \quad (3)$$

therefore we set the differential system:

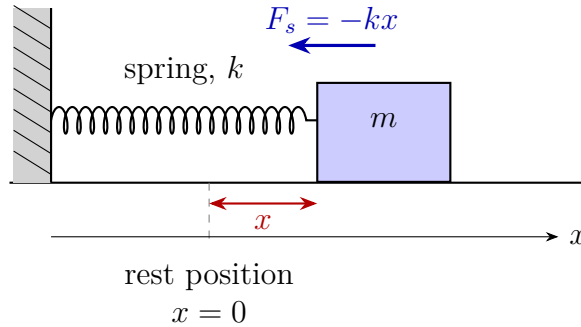
$$\begin{cases} \vec{F}(t) = m\vec{a}(t) = m\frac{d}{dt}(\vec{v}(t)) \\ \vec{v}(t) = \frac{d}{dt}(\vec{r}(t)) \\ \vec{v}(0) = \vec{v}_0 \\ \vec{r}(0) = \vec{r}_0 \end{cases}$$

$$\Rightarrow m\frac{d^2}{dt^2}(\vec{r}(t)) = F(\vec{r}, t)$$

2.1 Hooke's Law

A classical example in Newtonian mechanics is the motion of a mass attached to a spring. The restoring force exerted by the spring is proportional to the displacement from the equilibrium position and is directed toward the equilibrium itself.

Let $x(t)$ be the displacement of the mass from equilibrium along one axis.



Hooke's law states:

$$F(t) = -kx(t)$$

where:

- $k > 0$ is the spring constant
- $x(t)$ is the displacement from equilibrium
- the minus sign indicates that the force acts in the opposite direction of the displacement

Applying Newton's second law, we obtain:

$$m\ddot{x}(t) = -kx(t)$$

therefore the differential equation of motion is:

$$m\ddot{x}(t) + kx(t) = 0$$

We complete the model with the initial conditions:

$$x(0) = x_0, \quad \dot{x}(0) = v_0$$

Hence the problem can be written as:

$$\begin{cases} m\ddot{x}(t) = -kx(t) \\ x(0) = x_0 \\ \dot{x}(0) = v_0 \end{cases}$$

This is a second order linear homogeneous differential equation. Dividing by m , we get:

$$\ddot{x}(t) + \frac{k}{m}x(t) = 0$$

For the sake of calculation, if we define

$$\omega = \sqrt{\frac{k}{m}}$$

we can solve this equation by looking for solutions of the form:

$$x(t) = e^{\lambda t}$$

Substituting into the differential equation, we obtain:

$$\lambda^2 e^{\lambda t} + \omega^2 e^{\lambda t} = 0$$

Since $e^{\lambda t} \neq 0$, this gives the characteristic equation:

$$\lambda^2 + \omega^2 = 0$$

thus:

$$\lambda = \pm i\omega$$

Therefore the general solution is:

$$x(t) = c_1 e^{i\omega t} + c_2 e^{-i\omega t}$$

Using Euler's formula, this can also be written in real form as:

$$x(t) = A \cos(\omega t) + B \sin(\omega t)$$

Imposing the initial conditions:

$$\begin{aligned} x(0) &= A = x_0 \\ \dot{x}(0) &= B\omega = v_0 \end{aligned}$$

hence:

$$B = \frac{v_0}{\omega}$$

Therefore the solution satisfying the initial conditions is:

$$x(t) = x_0 \cos(\omega t) + \frac{v_0}{\omega} \sin(\omega t)$$

This is called simple harmonic oscillator: the mass oscillates periodically around the equilibrium position with angular frequency $\omega = \sqrt{\frac{k}{m}}$.

3 Lagrangian Mechanics

Solve the dynamics of a system using only algebraic procedures to get the same second order differential equation system coming from Newtonian mechanics – but it is general even to non relativistic mechanics.

3.1 Mass Matrix

Before setting the Lagrangian system, we need to define the *mass matrix* so that we can generalize the concept of **kinetic energy**. We define the kinetic energy as a scalar quantity in the i -th direction:

$$d\mathcal{T}_i = dr_i F_i = dr_i m_i \ddot{r}_i = \frac{dr_i}{dt} m_i d\dot{r}_i ,$$

that we can rewrite using $d(\dot{r}_i \cdot \dot{r}_i) = 2\dot{r}_i d\dot{r}_i$ as:

$$\begin{aligned} d\mathcal{T}_i &= m_i \frac{1}{2} d(\dot{r}_i^2) \\ \Rightarrow \mathcal{T}_i &= \int_{\dot{r}_0}^{\dot{r}_1} \frac{1}{2} m_i d(\dot{r}_i^2) = \frac{1}{2} m_i (\dot{r}_{i,1}^2 - \dot{r}_{i,0}^2) = \frac{1}{2} m_i \dot{r}_i^2 \end{aligned}$$

We can write all the contributions to the energy in a vectorial form:

$$\mathcal{T} = \frac{1}{2} \dot{\vec{r}}^\top M(\vec{r}, t) \dot{\vec{r}} ,$$

with $M(\vec{r}, t)$ being the **mass matrix**. In the euclidean coordinates, $M(\vec{r}, t) \equiv M_{eu}$ is diagonal filled with the single masses – the difficulty is in defining the velocities. Using the generalized coordinates instead $\vec{r} = \vec{r}(\vec{q})$ we get:

$$\frac{d\vec{r}(\vec{q})}{dt} = \sum_i \frac{\partial \vec{r}_k}{\partial q_i} \dot{q}_i + \frac{\partial \vec{r}(\vec{q})}{\partial t} = J_q \dot{\vec{q}} + \frac{\partial \vec{r}(\vec{q})}{\partial t} = \dot{\vec{r}}(\vec{q}) .$$

Substituting this into E and reordering the terms we get:

$$\mathcal{T} = \frac{1}{2} \left(\dot{\vec{q}}^\top M(\vec{q}, t) \dot{\vec{q}} + \vec{M}_v(\vec{q}, t) \cdot \dot{\vec{q}} + M_0(\vec{q}, t) \right) ,$$

where the details are the following:

$$M(\vec{q})_{i,j} = \sum_k m_k \frac{\partial \vec{r}_k}{\partial q_i} \frac{\partial \vec{r}_k}{\partial q_j} = \left(J_q^\top M_{eu} J_q \right)_{i,j} \longrightarrow \text{Generalized inertia}$$

$$\vec{M}_v(\vec{q})_i = \sum_k m_k \frac{\partial \vec{r}_k}{\partial q_i} \frac{\partial \vec{r}_k}{\partial t} = \left(J_q^\top M_{eu} \frac{\partial \vec{r}}{\partial t} \right)_i \longrightarrow \text{Linear velocity coefficient}$$

$$M_0(\vec{q}) = \sum_k m_k \left(\frac{\partial \vec{r}_k}{\partial t} \right)^2 \longrightarrow \text{Time dependent potential-like term}$$

which for the systems in which the coordinates transformation is not affected by the time, then we have:

$$\mathcal{T} = \frac{1}{2} \dot{\vec{q}}^\top M \dot{\vec{q}} . \tag{4}$$

Clearly if we are dealing with points of the same mass (or a single point) $M = m_{tot} I$, with I being the identity matrix.

3.2 Lagrange's Equations

We start by defining one of the most profound empirical observations in physics: the **principle of Least Action**. This fundamental principle states that, for any dynamical system, the path followed by the system between two states is the one that minimizes a quantity called **action** S , which is defined as:

$$S = \int_{t_0}^{t_1} \mathcal{L}(\vec{q}, \dot{\vec{q}}, t) dt , \quad (5)$$

where $\mathcal{L}(\vec{q}, \dot{\vec{q}}, t)$ is a specific function embedding the dynamics of the system. This function is called **Lagrangian** and it must be a function of $\vec{q}$ and $\dot{\vec{q}}$ the generalized coordinates describing the system.

The action depends on the path followed by the system, so we cannot write it as a differential. However, if we consider small variations in the path – not time – keeping the endpoints fixed we can write:

$$\delta S = \int_{t_0}^{t_1} \left(\frac{\partial \mathcal{L}}{\partial \vec{q}} \delta \vec{q} + \frac{\partial \mathcal{L}}{\partial \dot{\vec{q}}} \delta \dot{\vec{q}} \right) dt . \quad (6)$$

Integrating by parts the second term:

$$\begin{aligned} \delta S &= \int_{t_0}^{t_1} \frac{\partial \mathcal{L}}{\partial \vec{q}} \delta \vec{q} dt + \left[\frac{\partial \mathcal{L}}{\partial \dot{\vec{q}}} \delta \vec{q} \right]_{t_0}^{t_1} - \int_{t_0}^{t_1} \frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\vec{q}}} \right) \delta \vec{q} dt = \\ &= \int_{t_0}^{t_1} \left(\frac{\partial \mathcal{L}}{\partial \vec{q}} \delta \vec{q} - \frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{\vec{q}}} \right) \delta \vec{q} dt \end{aligned}$$

where we used the fact that $\delta \vec{q}(t_0) = \delta \vec{q}(t_1) = 0$, since the endpoints are fixed. For the principle of least action to hold, we need $\delta S = 0$ for any arbitrary variation $\delta \vec{q}$. This is possible only if the integrand is zero, thus:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\vec{q}}} \right) - \frac{\partial \mathcal{L}}{\partial \vec{q}} = 0 , \quad (7)$$

which is the **Lagrange's equation**.

Now we want to find a relation between the Lagrangian $\mathcal{L}(\vec{q}, \dot{\vec{q}}, t)$ with the known Newtonian mechanics. We observe a similarity between the Lagrange's equation and the Newton's second law:

$$\begin{aligned} m \ddot{\vec{r}} &= -\nabla V(\vec{r}, t) \\ \frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\vec{q}}} \right) &= \frac{\partial \mathcal{L}}{\partial \vec{q}} \\ \Rightarrow \begin{cases} \frac{\partial \mathcal{L}}{\partial \vec{q}} \sim -\nabla V(\vec{r}, t) \\ \frac{\partial \mathcal{L}}{\partial \dot{\vec{q}}} \sim m \dot{\vec{r}} \end{cases} \end{aligned}$$

From this analogy, we can define the Lagrangian as:

$$\mathcal{L}(\vec{q}, \dot{\vec{q}}, t) = \mathcal{T}(\vec{q}, \dot{\vec{q}}, t) - V(\vec{q}, t) + g(t) , \quad (8)$$

where $\mathcal{T}$ is the kinetic energy expressed in generalized coordinates, V is the potential energy of the system and $g(t)$ is a generic function of time that doesn't affect the dynamics. With this definition, we can recover exactly the Newtonian dynamics using the Lagrange's equation, avoiding all the geometrical considerations about forces.

This definition of Lagrangian is very general, and holds also for relativistic mechanics if we define all the quantities properly.

4 Hamiltonian Mechanics

Hamiltonian mechanics arises from Lagrangian mechanics by rewriting the second order differential equation as a system of first order differential equations. In particular, for this purpose we use the **Legendre transform** as follows:

$$\begin{aligned} L_T(\mathcal{L}(q_i, \dot{q}_i, t))(q_i, p_i, t) &= \sum_i \dot{q}_i \frac{\partial \mathcal{L}}{\partial \dot{q}_i} - \mathcal{L}(q_i, \dot{q}_i, t) \\ \Rightarrow \mathcal{H}(q_i, p_i, t) &= \sum_i \dot{q}_i p_i - \mathcal{L}(q_i, \dot{q}_i, t) \end{aligned}$$

The dynamics of the system is now described by the system:

$$\begin{cases} \frac{dq_i}{dt} = \frac{\partial \mathcal{H}}{\partial p_i} \\ \frac{\partial \mathcal{L}}{\partial q_i} = \frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i} \right) = \frac{d}{dt}(p_i) = -\frac{\partial \mathcal{H}}{\partial q_i} \end{cases}$$

which gives the Hamilton equations:

$$\begin{cases} \frac{dq_i}{dt} = \frac{\partial \mathcal{H}}{\partial p_i} \\ \frac{dp_i}{dt} = -\frac{\partial \mathcal{H}}{\partial q_i} \end{cases}$$

Hamilton equations are crucial when describing the dynamics of a mechanical system because they are directly connected to the concept of **phase space**. The phase space is the (p, q) plane that encapsulate all the dynamic of the system in terms of energy and possible trajectories. In addition, almost the totality of numerical integrators requires first order differential equations, which is exactly what we obtained from the Legendre transform.

4.1 Phase Space

The phase space is an extremely useful tool to extrapolate quantitative/qualitative information about dynamical systems, in particular about the energy of systems. Once we managed to write the Hamiltonian of the system $\mathcal{H}(\vec{q}, \vec{p})$ we can apply hamilton equation to get $\dot{\vec{q}}$ and $\dot{\vec{p}}$ and solve the dynamics of the system.

The graph in the phase plane can be usually drawn directly from $\mathcal{H}(\vec{q}, \vec{p})$ by writing p in function of q .

In the example below we see:

$$\mathcal{H}(\vec{q}, \vec{p}) = \frac{\vec{p}^2}{2} + \frac{\vec{q}^2}{2}$$

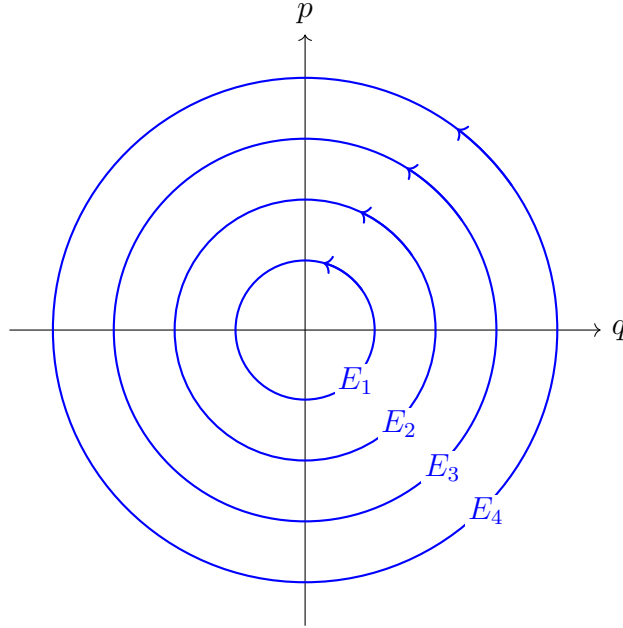


Figure 2: Phase-space trajectories of the harmonic oscillator for different energies.

starting from different initial conditions (which means different energies). Each closed curve in the phase space corresponds to a specific energy configuration of the system.

5 Statistical Mechanics

Statistical Mechanics is the main tool to describe high degrees of freedom systems, in particular by extracting generic properties from statistical considerations. Many fields are based on statistical mechanics, from material physics to social network analysis.

Assume to have a system s that can occupy discrete energy states ε_i . The system s interacts with a reservoir R , so that the total energy $E_{tot} = \varepsilon_i + \varepsilon_R$.

5.1 Canonical Ensemble

In the canonical configuration, the systems s and R exchange only energy, not matter. This means that each energy ε_i can be obtained in $g(\varepsilon_i)$ number of ways. The function $g(\varepsilon)$ is called **degeneracy** of the system s . The reservoir degeneracy is $g_R(E_{tot} - \varepsilon_i)$. We assume that all the states i of the whole system $s + R$ at an energy E_{tot} are *equally probable*, which is reasonable.

Since R is large compared to s , any change in R is a smooth function of ε , which means that we do not have sharp jumps in the physical quantities.

We define a quantity called **entropy** as:

$$S(E) = k_B \log(g(E))$$

which is a transformation of the degeneracy, in other words it is a measure of the probabilistic configuration of the system. With the entropy being defined, we can invert the relation:

$$g(E) = \exp\left(\frac{S(E)}{k_B}\right),$$

which means that the number of possible states at an energy ε_i for the whole system $s + R$ is:

$$N(\varepsilon_i) = g(\varepsilon_i) g_R(E_{tot} - \varepsilon_i) .$$

Since each microstate has the same probability to occur, that will be proportional to:

$$P(\varepsilon_i) \propto N(\varepsilon_i) = g(\varepsilon_i) , \exp\left(\frac{S(E_{tot} - \varepsilon_i)}{k_B}\right) ,$$

and since we know that $s \ll R$ we can expand with Taylor $S(E_{tot} - \varepsilon_i)$ around $(E_{tot} - \varepsilon_i) \sim E_{tot}$:

$$S(E_{tot} - \varepsilon_i) = S(E_{tot}) - \varepsilon_i \left. \frac{dS(E)}{dE} \right|_{E_{tot}} + \frac{\varepsilon_i^2}{2} \left. \frac{d^2 S(E)}{dE^2} \right|_{E_{tot}} + \dots$$

If we call the quantity

$$\left. \frac{dS(E)}{dE} \right|_{E_{tot}} \equiv \frac{1}{T}$$

keeping only the first order term we can rewrite the probability $P(\varepsilon_i)$ as:

$$\begin{aligned} P(\varepsilon_i) &\propto g(\varepsilon_i) \cdot \exp\left(\frac{S(E_{tot})}{k_B}\right) \cdot \exp\left(\frac{-\varepsilon_i}{k_B T}\right) \cdot \dots \\ \Rightarrow P(\varepsilon_i) &\propto g(\varepsilon_i) \exp(-\beta \varepsilon_i) , \quad \beta = \frac{1}{k_B T} \end{aligned}$$

and since the relation $\sum_i P(\varepsilon_i) = 1$ must hold, we get that the probability to find the system in an energy configuration ε_i is:

$$P(\varepsilon_i) = \frac{g(\varepsilon_i) \exp(-\beta \varepsilon_i)}{\mathcal{Z}(\beta)} , \quad \mathcal{Z}(\beta) = \sum_j g(\varepsilon_j) \exp(-\beta \varepsilon_j)$$

where $\mathcal{Z}(\beta)$ is called **canonical partition function** and it's the normalization coefficient of the probability.

For dynamical systems we use the **Hamiltonian** as the energy:

$$\mathcal{Z}(\beta) = \sum_j g(\varepsilon_j) \exp(-\beta \mathcal{H}_i(\vec{q}, \vec{p}))$$

5.2 Grand Canonical Ensemble

...

6 Fourier Analysis

Fourier formalism is based on the concept that L^2 has a basis of periodic functions $\{e^{inx}\}$ in an interval $[-T/2, T/2]$, which means that:

$$\int_{-T/2}^{T/2} e^{inx} e^{imx} = T \delta_{mn} ,$$

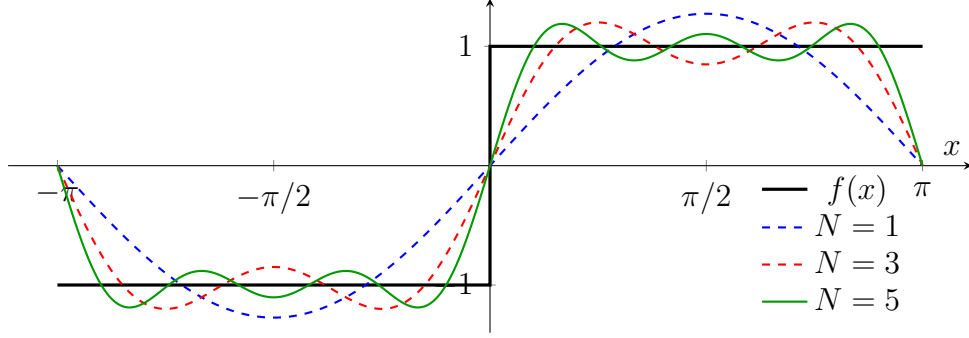


Figure 3: Example of a Fourier expansion: a periodic square wave $f(x)$ and its partial sums. By adding more Fourier modes, the series reproduces the function more accurately.

so that we can actually write any function $f \in L^2$ as:

$$f(x) = \sum_{n=-\infty}^{+\infty} F_n e^{inx}.$$

The values of the coefficients F_n can be found as:

$$\begin{aligned} \int_{-T/2}^{T/2} dx f(x) e^{-imx} &= \sum_{n=-\infty}^{+\infty} F_n \int_{-T/2}^{T/2} dx e^{inx} e^{-imx} = T F_m \\ \Rightarrow F_n &= \frac{1}{T} \int_{-T/2}^{T/2} dx f(x) e^{inx}, \end{aligned}$$

where T is the periodicity of the function. F_n are the amplitudes of the waves generating $f(x)$, so in other words the contribution to f given by a specific frequency.

We now want to take the limit for the period T that goes to infinity. We rewrite the decomposition of f within T and we take the limit $T \rightarrow +\infty$. Considering the limit, we

Quantity	T -finite	$T \rightarrow +\infty$
Allowed Frequencies	$k_n = \frac{2\pi n}{T}$	$k \in \mathbb{R}$
Frequency Distance	$\Delta k = \frac{2\pi}{T}$	dk
Amplitude Coefficients	$F_n = \frac{1}{T} \int_{-T/2}^{T/2} dx f(x) e^{ik_n x}$	$F(k) = \lim_{n \rightarrow +\infty} \frac{F_n}{T}$
Function Decomposition	$f(x) = \sum_{n=-\infty}^{+\infty} F_n e^{ik_n x}$	$f(x) = \int_{-\infty}^{+\infty} dk F(k) e^{ikx}$

can finally explicitly write the amplitude coefficients:

$$\begin{aligned} F_n &= \sum_{n=-\infty}^{+\infty} F_n e^{ik_n x} \cdot \frac{\Delta k}{\Delta k} = \sum_{n=-\infty}^{+\infty} \left(F_n \frac{T}{2\pi} \right) e^{ik_n x} \Delta k \\ &\rightarrow F(k_n) \equiv F_n \frac{T}{2\pi} = \frac{1}{T} \int_{-T/2}^{T/2} dx f(x) e^{-ik_n x} \cdot \frac{T}{2\pi} \xrightarrow{T \rightarrow +\infty} \frac{1}{2\pi} \int_{-\infty}^{+\infty} dx f(x) e^{-ikx} \\ &\Rightarrow F(k) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} dx f(x) e^{-ikx} \end{aligned}$$

which is the **Fourier Transform** of the function f . In other words, $F(\nu)$ is the contribution of the frequency ν when generating the function $f(x)$. This is used in any field of science, from signal processing, to quantum mechanics.

7 Information Theory

Information theory aims to quantify the concept of "information" of a system configuration. Suppose to have a set of events $\{x_i\}$ each with a probability $P(x_i) = P_i$ to occur. We want to find a quantity related to x_i such that it must be *non negative, higher for rare events* and *additive*. We define **self-information** of x_i the quantity:

$$I(x_i) = -k \log(P(x_i))$$

and we notice that

- $I(P) \leq 0, \forall P \in [0, 1]$
- $I(P_1) > I(P_2)$, if $P_1 > P_2$
- $I(P_1 \cdot P_2) = I(P_1) + I(P_2)$

$I(P)$ gives information about a single element. If we have a set $\{x_i\}$ then it's natural to find the *expected information value* if a generic element of the set as:

$$E[I(P(x))] = \sum_i P_i I(P(x_i)) = -k \sum_i P(x_i) \log(P(x_i)) \equiv S(x) .$$

The quantity $S(x)$ is called **Shannon entropy** and we note that it satisfies:

- **Additivity** : $S(x, y) = E[I_x, I_y] = E[I_x] + E[I_y] = S(x) + S(y)$
- **Increasing Monotonically** : $S(\{x_i\}) \leq S(\{x_i\}^n + x_{n+1})$ (so if we add an element to the set, the total entropy increases)
- **Maximal for uniform probabilities** : $S(\{x_i\}_{\text{uniform}}) \geq S(\{x_i\}_{\text{any}})$, $P_i = \frac{1}{N}$
- **Continuous in P** : Any change in $P(x_i)$ will affect S accordingly.

Also, Shannon uniqueness theorem tells us that $S(x)$ is the only function that satisfies all the four conditions. It's useful to know that the base of the logarithm gives us the possibility to model the system information on the specific problem. For instance, in the case of PC programming, where we store data in bits (0,1), we use the base 2 logarithm so that $I(x_i) = \log_2(P(x_i))$ is the number of bytes required to store x_i . This also implies that $S_{\max}(x)$ is the maximum compression obtainable to store the sequence of elements $\{x_i\}$.

8 Game Theory

8.1 Shapley Value

Shapley value is an algorithm to estimate the proper contribution of a generic player in a specific game. The game should have a "score" function that gives a quantitative output.

$$\psi(p, \mathbf{f}) = \frac{1}{|N|!} \sum_{C_p \in P(N_p)} (\mathbf{f}(C_p \cup \{p\}) - \mathbf{f}(C_p))$$

where $\psi(p, \mathbf{f})$ is the Shapley value of the element p considering $\mathbf{f}$ as the score function of the game, $N_p = N \setminus \{p\}$ is the set of players in the game excluded p , $P(N_p)$ is its power set and C_p is an element of the power set.

This algorithm is widely used also in machine learning in order to estimate the relevance of a feature during a model prediction. It works great with tree-based models.

9 Network Theory

We define $G(V, E)$ as a graph. V is the set of nodes, E is the set of edges. The number of nodes is $|V| = N$, while the number of edges is $|E| = m$.

9.1 Types of Matrixes

Adjacency Matrix

We define adjacency matrix A the structure that identifies if a link exists:

$$A_{i,j} = \begin{cases} 1, & \text{if } (i, j) \in E \\ 0, & \text{otherwise} \end{cases}$$

For undirected graphs, $A = A^T$, which is clearly not true for directed ones. For weighted graphs, the adjacency matrix becomes W , where we simply insert the edge weight attribute instead of 0/1.

Degree Matrix

We define the degree matrix as D :

$$D_{i,j} = \begin{cases} \sum_{k=1}^N A_{i,k}, & \text{if } i=j \\ 0, & \text{if } i \neq j \end{cases}$$

which is a diagonal matrix. The same definition considering the weighted adjacency matrix allows us to find the weighted degree matrix D_W .

Now, in the definition we wrote we slide over k , keeping i fixed. This is ok for symmetric adjacency matrixes, since $A_{i,j} = A_{j,i}$, but not for directed graphs! In fact, we expand our definition with an **in-degree** and **out-degree** matrix

$$D_{i,j}^{out} = \begin{cases} \sum_{k=1}^N A_{i,k}, & \text{if } i=j \\ 0, & \text{if } i \neq j \end{cases}$$

$$D_{i,j}^{in} = \begin{cases} \sum_{k=1}^N A_{k,j}, & \text{if } i=j \\ 0, & \text{if } i \neq j \end{cases}$$

These two concepts are crucial for steady state probabilities (page rank, diffusion, etc.). Usually, we use the notation ν_u to identify the degree of the node u .

Neighborhood

Given a graph $G(V, E)$, the neighborhood N of a node u is defined as

$$N(u) = \{i \in V \mid (u, i) \in E\} \cup \{u\}$$

If we remove the node itself we get the neighbors nodes of u .

Transition Matrix

For an unweighted graph, we define the transition matrix (or Markov matrix) as

$$P_{i,j} = \frac{A_{i,j}}{\sum_k A_{i,k}} = (D^{-1}A)_{i,j}$$

Similarly, for an weighted graph, we define the transition matrix as

$$P_{i,j} = \frac{W_{i,j}}{\sum_k W_{i,k}} = (D_W^{-1}W)_{i,j}$$

Again, we pay attention to the fact that we are not specifying if we are evaluating the *in* or *out* degree. Both have a well defined physical meaning, one is the probability to get to the node, one to leave the node. It's more intuitive the out-degree transition matrix, since the other has like a backward logic.

It's important to notice that the transition matrix rows (for out-degree) or columns (for in-degree) must add up to 1

$$\sum_i^N P_{i,j}^{in} = 1$$
$$\sum_j^N P_{i,j}^{out} = 1$$

This will allows us to exploit some important theorems for markov matrixes.

Path Matrix

Once we know the adjacency matrix A , we can actually take any power l of that to get the number of paths of length l between a pair of the nodes

$$A_{i,j}^l = \text{number of paths of length } l \text{ between } i \text{ and } j$$

This can be easily shown by taking the matrix power

$$(A^l)_{i,j} = \sum_{k=1}^N A_{i,k}^{l-1} \cdot A_{k,j}$$

which is equal to 0 if there is no path $i \rightarrow k$ in the graph identified by A^{l-1} and also if there is no path $k \rightarrow j$ in the graph identified by A (the starting network), otherwise it's just a sum of ones (since A is a 0/1 matrix).

Now, if we iterate this l times we clearly see that, in the end, we are tracing all the paths of length l ($u \rightarrow v \rightarrow \dots \rightarrow i \rightarrow k \rightarrow j$) possible in the network. This shows an elegant property of the adjacency matrix, with some important implications (like cycle finding on the diagonal).

It is also worth noticing that we are doing no assumption on the adjacency matrix symmetry, so this is valid for both directed and undirected networks.

Sparse Matrix Formalism

todo, formalism to use to save memory when dealing with matrixes with many zeros. dont know how to use it in practice tho.

Hub

Hub is an informal way to identify an highly connected node in a network. This concept emerges frequently in peer reviews and social network analysis. It is usually used in **directed** networks to identify a node with high in-degree, without considering the out-degree.

In fact, an hub can be an editor (i.e. PubMed), an influencer (i.e. Cristiano Ronaldo), but if we define the network according to our use case, an hub can be a professor, a parent etc.. Ideally, all the nodes are hubs, that's why the definition is not rigorous, however we can determine how much a node behaves like an hub.

NOTE: In Kleinberg centrality we will introduce the concept of **authority**. For Kleinberg authority is what we mean by hub, and hub is a node that points to many good authorities, which is just stupid, so I will always invert them. Sadly, network theory is quite a new field, so it hasn't developed a properly unified nomenclature. Just the name itself, graph or network, is astonishingly stupid (cause a graph is a damn graph, and a network is waaay to many things).

9.2 Types of Networks

Here we present some of the recurrent network structures that appear in letterature and that have been studied. We always consider a generic graph $G(V, E)$.

- **Complete Graph** : All the nodes are connected to every other node (K_N).
- **Regular Graph** : All the nodes have the same degree (K_n).
- **Cycle Graph** : All the nodes have degree 2 and forms a perfect cycle.
- **Tree Graph** : Acyclic graph and has $m = N - 1$ edges (a star graph is a special case of tree).
- **Bipartite Graph** : Graph that has two different set of nodes V_1 and V_2 , each with no inner connection between the nodes. A tree is also bipartite. We can also have n -partite graphs, but usually it's only convenient the bipartite.
- **Complete Bipartite** : Graph that has two different set of nodes V_1 and V_2 , each with no inner connection between the nodes and every node of set V_1 is connected to all the nodes of V_2 and viceversa.
- **Lattice Graph** : Network that can be arranged in a regular grid, so that forms a regular tiling if embedded into an euclidean space.

Now we present *toy networks* that are usually used to randomly generate graphs. There are others, but a these two are the only ones that can resable something useful in the real world.

- **Erdős–Rényi Graph** : Random graph, each node with a probability p to have a link with any other.
- **Barabási–Albert** : Iterative graph, that exhibits power law behavior. Each time, add a new node u with a probability $p_v = \nu_v / \sum \nu_j$ to be connected with the node v .

9.3 Centralities

Centrality measures are the most useful method to get both local and global information regarding nodes and its neighbors. There are many of them out there, all serving different purposes. In general, networks are very complex structures and we may don't have a general recipe to complete a task, or extracting specific information. It's usually a trade-off between accuracy required and performance costs. All the following discussions will consider a network $G(V, E)$, with N number of nodes and m number of edges.

Degree Centrality

Degree centrality is by far the easiest yet surprisingly effective centrality there is. It is defined as

$$\vec{C}_d = \frac{1}{N-1} D \vec{1}$$

where D is the unweighted degree matrix and $\vec{1}$ is the unit vector. It can be defined both for D^{out} and D^{in} , and it shows how much a node is connected in the network.

If we consider the weighted degree matrix D_W , we get the *strength centrality*, which combined with the degree centrality can give us an insight on the most effective nodes existing in the network, or precious hints on finding the potential ones (i.e. a node with many weak connections, if improved, can become crucial).

Betweenness Centrality

Betweenness centrality is a quantity used to determine the most crucial nodes for the network connectivity, in other words, those nodes that are visited by the most number of shortest paths. The definition of betweenness centrality C_b for a node u

$$C_b(u) = \sum_{v \neq u \neq w} \frac{N_{sp}(u, w|u)}{N_{sp}(u, w)}$$

where $N_{sp}(a, b)$ gets the number of shortest paths between the nodes a and b , and $N_{sp}(a, b|c)$ requires those paths to pass from c . The complexity is clearly high, in fact it scales as $O(N^3)$ for a naive solution, and $O(N \cdot m)$ in case of unweighted graphs, or $O(N \cdot m + N^2 \cdot \log N)$ for positive weights. Usually, we use approximated methods, mostly sampling based methods.

Closeness Centrality

Closeness centrality quantifies how near a node is to all other nodes in a network. It reflects how efficiently information can spread from that node to others and can, for

instance, support recommendation systems based on the proximity of nodes' interests. It is defined as:

$$C_c(u) = \frac{N - 1}{\sum_{v \neq u} d(v, u)}$$

where $d(v, u)$ is the length of the shortest path between v and u (the geodesic distance). It's important to notice that, for directed networks, $d(u, v) \neq d(v, u)$, so we have to keep in mind what we are asking. This centrality is defined only for connected graphs, since $d(u, v) = +\infty$ for disconnected components.

Harmonic Centrality

Harmonic centrality also measures how close a node is to all others but provides a more robust formulation for disconnected networks. It is defined as:

$$C_c(u) = \sum_{v \neq u} \frac{1}{d(v, u)}$$

where $d(v, u)$ is the length of the shortest path between v and u (the geodesic distance). Again, notice that for directed networks, $d(u, v) \neq d(v, u)$. This centrality addresses the problem of closeness centrality for unconnected graphs, where $d(u, v) = +\infty$ would get all to zero. The use cases are basically the same as the closeness (in reality, there is no use to use closeness, just use this)

Eigenvector Centrality

This centrality is a bit tricky, so let's start by recalling the eigenvector problem, in terms of adjacency matrix, and $\vec{x}$ being an eigenvector

$$A\vec{x} = \lambda\vec{x}$$

Now let's recall a few theorems. First, **Perron-Frobenius theorem** assures us that, if A is a real square matrix, then the eigenvector associated to the largest eigenvalue has all positive components. This is interesting if we want to exploit this for a centrality measure (it should be a positive quantity for physical interpretation).

So, let's assume that A is an adjacency matrix for a weighted graph with non negative weights. At this point, we can evaluate the largest eigenvector solving the eigenvalue problem

$$A\vec{x}_L = \lambda_L\vec{x}_L$$

and we can rewrite this emphasizing the i -th component of the largest eigenvector $\vec{x}_L$ as

$$x_{L,i} = \frac{1}{\lambda} \sum_j A_{i,k} x_{L,j}$$

By definition, we evaluated the eigenvector centrality measure of the i -th node. Now, we can give it a physical meaning as follows: The eigenvector centrality quantify how much a node is important, and how important its neighborhood is too, which affects the total score. We can then rename for consistency

$$\vec{x} = \vec{C}_e$$

This centrality is quite easy to compute, and it tells particularly well the *rank* system of the nodes in the network. In particular, it is used for *undirected* networks.

Kleinberg Centrality

Kleinberg centrality tackles the problem of recognizing the importance of hubs and authorities in directed graphs. As we introduced earlier, *hubs* are nodes that have high in-degree connectivity, but we can improve this concept by quantifying the "quality" of these connections. Here we introduce the concept of *authority*, which are nodes that points to many "good" hubs. We define hubs $\vec{h}$ and authorities $\vec{r}$ as:

$$\begin{aligned}\vec{h} &= A^\top \vec{r} \\ \vec{r} &= A \vec{h}\end{aligned}$$

which in the case of an *undirected* network we have that $A^\top = A$, therefore it makes no sense considering $\vec{h} \neq \vec{r}$, thus we can impose $\vec{h} = \vec{r}$ and we proceed with the traditional eigenvalue problem.

$$\vec{x} = A \vec{x}$$

In this context, *hubs* are nodes that are pointed by many good authorities, while *authorities* are nodes that points many good *hubs*. In other words, authorities build bridges between good hubs.

This algorithm is used when dealing with *sub-networks* or query-dependent networks, like topic related graphs.

NOTE: To keep consistency with the initial definition of hubs, I inverted Kleinberg definition of hubs and authorities. So, whenever you find this algorithm, remember to switch them.

PageRank Centrality

PageRank algorithm is the eigenvector centrality with the addition of a random jumping factor. We have to keep in mind that we are studying a graph $G(V, E)$ with N nodes and m edges. The network G is described by the weighted adjacency matrix W .

We don't start from the eigenvalue problem, but we start by setting up the node presence probability equation. We define the page rank equation:

$$\vec{C}_{pr} = \alpha P^{in} \vec{C}_{pr} + (1 - \alpha) \vec{v}$$

which states exactly: *Given any node i , node i PageRank score depends on which other nodes point to i^* , and we find it as a steady-state probability. We use the steady state probability because we want to study the equilibrium state, otherwise $\vec{C}_{pr} \in \mathbb{R}^N$ would be different from left and right.

The stochastic vector $\vec{v} \in \mathbb{R}^N$ is exactly the probability of jumping to a node in the network. In fact, $(1 - \alpha)$ is the probability to jump, but where to jump is given by $\vec{v}$. Usually, we simply define the stochastic vector as

$$\vec{v}_i = \frac{1}{N}$$

We recall the definition of the *in-degrees* stochastic matrix $P \in (\mathbb{R}^N \times \mathbb{R}^N)$ (we will drop the "in" later):

$$P_{i,j}^{in} = \frac{1}{\sum_k A_{k,j}} = \frac{W_{i,j}}{\sum_k W_{k,j}}$$

Each column adds up to 1, which is fundamental for *Perron-Frobenius theorem* in the case of normalized stochastic matrixes.

With all of this being said, using the fact that $\vec{1}^\top \vec{C}_{pr} = 1$, we can manipulate the page rank equation so that we can write:

$$\begin{aligned}\vec{C}_{pr} &= \alpha P \vec{C}_{pr} + (1 - \alpha) \vec{v} (\vec{1}^\top \vec{C}_{pr}) = \\ &= [\alpha P + (1 - \alpha) \vec{v} \vec{1}^\top] \vec{C}_{pr}\end{aligned}$$

Thus we get the Google Matrix

$$G = \alpha P + (1 - \alpha) \vec{v} \vec{1}^\top$$

with which we can set the eigenvalue problem

$$G \vec{C}_{pr} = \vec{C}_{pr}$$

where *Perron-Frobenius theorem* assures us that for a stochastic matrix, the largest eigenvalue is $\lambda = 1$, precisely our case.

From here on, it's just the eigenvector centrality evaluation. PageRank is used for ranking the pages importance. We must note that W is not required to be symmetric, just stochastic. This means that PageRank algorithm is perfect for *directed networks*. In particular, the random jump factor allows to manage also the *dead-end* nodes (no outgoing link), that cannot be treated by the simple eigenvector centrality.

Katz Centrality

Katz centrality consider the relevance of a node according to the total number of walks that pass through a given node. The definition is straight forward:

$$C_K(u) = \sum_{k=1}^{+\infty} \sum_{i=1}^N \beta^k (A^k)_{i,u}$$

which finds all the possible paths to u of all possible lengths. The value β is called attenuator factor, and is there to grant us the convergence of the series. In fact, it cannot be any number. For all the nodes we can use the unit vector $\vec{1}$:

$$\vec{C}_K = \sum_{k=1}^{+\infty} \beta^k (A^k)^\top \vec{1} = \sum_{k=1}^{+\infty} (\beta A^\top)^k \vec{1}$$

and at this point we recall Neumann Matrix expansion:

$$(I - A)^{-1} = \sum_{n=0}^{+\infty} A^n = I + \sum_{n=1}^{+\infty} A^n$$

with the spectral radius $\rho(A) < 1$ (the module of the maximum eigenvalue). We can rewrite Katz centrality as

$$\vec{C}_K = ((I - \beta A^\top)^{-1} - I) \vec{1}$$

with the requirement that

$$\beta < \frac{1}{\lambda_{max}}$$

in order for the series to converge.

9.4 Diffusion on Graphs (analytical)

The study of how a quantity will go through a structure is particularly interesting in many situations. This because it allows to predict the behavior of certain phenomena, like money distribution or power shortages.

Any diffusion process follows the diffusion equation

$$\frac{\partial}{\partial t} u(\vec{x}, t) = \nabla^2 (D(\vec{x}, t) \cdot u(\vec{x}, t))$$

which can be easily obtained by considering the continuity equation and the Fick's law

$$\frac{\partial}{\partial t} u(\vec{x}, t) + \nabla J(\vec{x}, t) = 0$$

$$J(\vec{x}, t) = -\nabla (D(\vec{x}, t) \cdot u(\vec{x}, t))$$

In a graph context, $\vec{u}$ are the quantities present in each node, and the diffusion equation can be written as

$$\frac{d}{dt} \vec{u}(t) = -L_D \vec{u}(t)$$

obtained as:

$$\begin{aligned} \frac{d}{dt} u_i(t) &= \sum_{j=1}^N W_{i,j} (u_j(t) - u_i(t)) = \\ &= \sum_{j=1}^N W_{i,j} u_j - u_i \sum_{j=1}^N W_{i,j} = \\ &= \sum_{j=1}^N W_{i,j} u_j - u_i D_i = \sum_{j=1}^N W_{i,j} u_j - \sum_{j=1}^N \delta_{i,j} u_i D_i = \\ &= \sum_{j=1}^N u_j (W_{i,j} - \delta_{i,j} D_i) \end{aligned}$$

thus we can consider all the nodes at the same time, obtaining the diffusion equation

$$\frac{d}{dt} \vec{u}(t) = (W - D) \vec{u}(t) = -L_D \vec{u}(t)$$

Note that here we explicitly consider the fact that $\vec{u}$ is a vector, since the concept of position in a network is different from the continuum space. Here each node is a single point, so we consider all at the same time.

The new matrix found L_D is called **Laplacian** matrix:

$$L_D = D - W$$

This reasoning is true for both *directed* and *undirected* graphs, with a very important difference. We have considered D at the node i , but without specifying if that is an "in-degree" or "out-degree" matrix. According to what we want to diffuse, we will need to specify it for directed networks.

9.5 Experimental Diffusion

In general, if we can't evaluate the Laplacian etc.. we do have **random walks** and **transition matrixes**. Random walks allow to simulate a diffusion process, according if we deploy a proper amount of independent walkers (probably central limit theorem or something like that). This is useful cause we can isolate a specific portion of the graph and perform many random walk simulations in order to monitor how the quantities changes. For Markov Ergodic Theorem (i think) we know that simulating infinite random walks on a given network, it will riproduce the steady state distribution.

Transition matrixes on the other hand provide a statistical description of where the quantities will go, according to the connectivity of the nodes. Both in the limit of many steps, will end up in the Laplacian case. The transition matrix, as we saw, is defined as

$$P_{i,j} = \frac{W_{i,j}}{\sum_k W_{i,k}} = (D_W^{-1}W)_{i,j}$$

again considering the incoming or outgoing links. We can use the transition matrix to perform step by step the diffusion of any quantity in a network by simply evaluating:

$$\vec{x}(t+1) = P \vec{x}(t)$$

which will eventually converge to the steady state distribution.

Cover Time

We define cover time τ as the expected number of steps required for a random walk to visit all the nodes in a network, starting from a random node u :

$$\tau = E[n_{steps}] = \lim_{n \rightarrow +\infty} \frac{\sum_{i=0}^n \tau_i}{n}$$

Hitting Time

We define hitting time $\vec{\tau}_u$ the expected number of steps required for a random walk to reach each node v in the network, starting from the node u :

$$\tau_{u,v} = E[n_{steps,i}|u] = \lim_{n \rightarrow +\infty} \frac{\sum_{i=0}^n n_i(v|u)}{n}$$

Note that $\tau_{u,v} \neq \tau_{v,u}$ even for undirected graph, in fact if we define the hitting time matrix τ it won't be symmetric unless the network is **regular** (all nodes have the same degree).

We can define the **commute time** as $C_{\tau_{u,v}} = \tau_{u,v} + \tau_{v,u}$ so that the commute matrix C_τ is actually symmetric.

9.6 Path Finding

Path finding is a fundamental topic in network theory and it is used in many fields, from mathematical modeling to video games characters behaviors. There are different algorithms, but A* is surely the most used.

Breath First Search (BFS)

Simple algorithm for graph traversal, in particular is used to find a path from a node u to a node v . We don't find shortest paths or stuff, just a path, depending on how you queue the nodes.

The idea is very simple: We define 2 structures, the **queue nodes** and the **visited nodes**, which are two different set of nodes. The *visited nodes* start empty, while the *queue nodes* start with the first node we want to visit.

- Start from node $u \rightarrow$ remove it from *queue*, add it to *visited*.
- Get the neighborhood $N(u)$ and insert every node in that set that is not in *visited* inside *queue*.
- Proceed with the next node w in *queue* $\rightarrow$ remove it from *queue*, add it to *visited*.
- Repeat 2 and 3.
- Stop when you find the required node v .

Dijkstra's Algorithm

Algorithm used to find the shortest path in a weighted graph. It adapts the BFS algorithm by providing the concept of **priority queue**, that selects the following node according to the minimum of distance from the starting one. We need some structures:

- **priority queue nodes** : where you select the node with smallest distance from the starting one. This is needed to get the shortest path length.
- **visited nodes** : set of visited nodes, and that will never analyzed again. This is needed to avoid loops.
- **previous nodes** : map between any node and its previous. The previous is chosen according to the smallest distance found (it will be update during the algorithm). This is needed to reconstruct the path.

We initialize all the distances to any node (except the starting one) with $+\infty$. The idea is:

1. We always visit the node in the priority queue. If a node is given by the priority queue, that node goes into the **visited nodes** and it will never be visited again. This assures that we reached the node in the smallest path possible, because of how the priority queue is constructed (prioritizes always the step with smallest distance, independently on which node we are).
2. Given a node i we evaluate the distance to all the neighbors nodes j so that we can get the distance from the starting node $d(u, j) = d(u, i) + W_{i,j}$. If $d(u, j)$ is smaller than the previous distance $d_{prev}(u, j)$ (recall that we initialized it to $+\infty$) then we update it.
3. We also assign the previous node to j , as the one with the smallest distance $d(i, j)$.

Once the priority queue gives us the target node v , then we stop. To reconstruct the path, we simply use the *previous nodes* map, starting from the *target node*, and arriving the starting one, ensuring the shortest path.

A* Algorithm

A* is the algorithm mostly used for path search, in videogames, gps, and many other fields. It is based on Dijkstra Algorithm, but it adds an *"educated"* estimation on how distant the target node is, so that it manages to avoid searching for evidently off paths (imagine going to a place, but moving in the opposite direction, it's pointless).

This works well if we are in a metric space in which the nodes are located, it's harder if we don't have a correspondence between nodes and metric place. In fact, GPS is the perfect example, since nodes are cities, and they are mapped to a specific point in the metric space.

We still initialize all the nodes distances with $+\infty$, but during the priority queue evaluation we also consider an additional term, the **heuristic distance**, from the incoming node and target node, exactly as a potential function. Assuming that u is the starting node and v is the target node, the priority queue will process a distance for the incoming node j :

$$f(j) = h(j, v) + d(u, j)$$

where $d(u, j)$ is the shortest distance from u to j .

There are many ways to define $h(x, y)$, usually we can use the *"euclidean distance"* if we are in $\mathbb{R}^N$, otherwise the *"arc"* distance if we are on an hypersphere.

9.7 Link Prediction

We can exploit similarities in order to predict nodes connections according to some pre-defined threshold. Clearly, each similarity express some specific node relation, the weights affects these relations too, so every case has its own peculiarities that must be taken in considerations.

Jaccard Similarity

Jaccard similarity is the fraction of elements in common between two neighborhoods, defined as:

$$S_J(u, v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|} \in [0, 1]$$

This gives a straightforward representation of similarity between two nodes, without considering any second order relations.

Adamic-Adar Similarity

Adamic-Adar is a refinement of Jaccard similarity. It quantifies the amount of elements somewhat in common between two nodes as:

$$S_{AA}(u, v) = \sum_{i \in (N(u) \cap N(v)) \setminus \{u, v\}} \frac{1}{\log(D_W(i))} \geq 0$$

with $D_W(i)$ being the value of the weighted degree matrix for the node i . Again we notice that D_W can represent both *"in"* or *"out"* degrees.

Random walk Similarity

Random walk similarity is a cool concept that can be obtained through simulations too. Given a node u in a graph $G(V, E)$ we define random walk similarity vector:

$$\vec{S}_R(u) = \vec{\tau}(u)^{-1}$$

with $\vec{\tau}(u)$ being the **hitting time** vector from u to all the nodes in the network. The hitting time is defined as the average number of steps required to get from a starting node u to a target node v .

10 Markov Processes

A Markov process is a stochastical sequence of events such that:

$$P(x_{n+1} = x | x_n, x_{n-1}, \dots, x_0) = P(x_{n+1} | x_n) ,$$

which means that the probability to have a new state x_{n+1} depends only on the current state in which my system is x_n . If the sequence of events $\{x_i\}$ is finite, we call it **Markov chain**, if instead $x \in \mathbb{R}$ then it's called **Brownian motion**.

We describe the Markov process using the **Transition matrix** T such that:

$$T : T_{i,j} = P(x_{n+1} = j | x_n = i) , \quad \sum_j T_{i,j} = 1 ,$$

so we are asking T to be *row-stochastic*. Given a generic system, containing many nodes with some quantities, the evolution of the system at a step t is:

$$\vec{s}_t = \vec{s}_0 T^t ,$$

where $t \in \mathbb{N}$ and $\vec{s}_0$ is the initial distribution of a generic quantity in a system. Clearly, the stationary distribution is obtained when nothing changes in the system when the transition matrix is applied:

$$\vec{s} = \vec{s} T$$

11 Time Series

Data can be collected in many different ways. Time series are a way to collect data by emphasizing the concept of "time" in which the data has been collected. This produces a list of peculiarities and difficulties that are a bit different from generic (x, y) points.

In broader terms, a time series is simply a collection of multidimensional points $\{(\vec{p}_i, t_i)\}_i$ where the order of acquisition t_i must be considered. This usually means that any point $\vec{p}_i$ depends in some way on the previous points, so $\{\vec{p}_i\}_i$ are **not independent**.

A **stationary time series**, or stationary process, is a series that do not change its statistical variables if shifted in times, meaning that mean, variance, behaviors, remain always constant. In real world application, time series are never stationary, but they can be useful as null hypothesis.

11.1 Theoretical Description

A time series is basically a Markov process $P(x_{n+1} = x | x_n, x_{n-1}, \dots, x_0) = P(x_{n+1} | x_n, \dots, x_{n-m})$ of order m , where in general the effect of further terms gets smaller. The value x_t is called *process*, and the set $\{x_t\}_0^n$ represents the steps taken in the process, or *path*.

The **expected value** of the process is:

$$\mu_t = E[x_t] = \int_{-\infty}^{+\infty} z f_t(z) dz ,$$

where $f_t(z)$ is the probability density distribution of x_t , z is the set of possible values that x_t can take. For a *stationary process* we have that $\mu_t = \mu$, $\forall t$.

The **variance** of the process is:

$$\sigma_{x,t} = \text{Var}[x_t] = E[(x_t - \mu_t)^2] ,$$

and again for a *stationary process* we have $\sigma_t = \sigma$, $\forall t$.

One very important quantity is the **autocorrelation**, because it allows to understand the influence of previous points on the newer ones:

$$R_x(t, j) = \frac{\text{Cov}[x_t, x_{t-j}]}{\sqrt{\text{Var}[x_t] \text{Var}[x_{t-j}]}} = \frac{E[(x_t - \mu_t)(x_{t-j} - \mu_{t-j})]}{\sqrt{\sigma_{x,t} \sigma_{x,t-j}}}$$

We understand that *white noise* $\{\varepsilon_t\}_t$ will contribute to these quantities as:

$$\begin{aligned} E[\varepsilon_t] &= 0 \\ \sigma_{\varepsilon,t} &= \sigma_\varepsilon \\ R_\varepsilon(t, j) &= 0, \forall t \neq j \end{aligned}$$

White noise by itself is a stationary process: its expected value is constant, its variance is constant, and its autocorrelation is zero for every non-zero lag. Therefore, adding white noise to a stationary time series does not necessarily destroy stationarity. For example, if

$$x_t = y_t + \varepsilon_t ,$$

where y_t is stationary and ε_t is independent white noise, then x_t is also stationary, with

$$\text{Var}[x_t] = \text{Var}[y_t] + \text{Var}[\varepsilon_t] .$$

The variance grows with time only when the white-noise terms are accumulated. In particular, consider the random walk:

$$x_t = x_0 + \sum_{k=1}^t \varepsilon_k .$$

Assuming that x_0 is fixed and that the white-noise terms are uncorrelated, we obtain:

$$\text{Var}[x_t] = \sum_{k=1}^t \text{Var}[\varepsilon_k] = t \sigma_\varepsilon^2 .$$

In this case, the variance depends on t and grows without bound, thus the random walk is not stationary. This means that it is not the presence of white noise itself that causes non-stationarity, but rather the inevitable white noise accumulation over time.

11.2 Technical Difficulties

In practical terms, time series are not that easy to work with. In particular, we can list a few known issues:

- **Temporal correlation:** Past events influence current and future observations. This violates the independence assumption used by many conventional machine-learning techniques, limiting the set of methods that can be applied directly and requiring time-aware validation and modelling approaches.
- **Changing trends and concept drift:** Time series often reflect dynamic business environments, such as CPG markets. Changes in commercial strategy, customer acquisition, pricing, distribution, promotions, or market conditions can alter the underlying data-generating process over time. As a result, patterns learned from historical data may no longer accurately describe future behaviour.
- **Timestamp quality and calendar alignment:** Raw timestamps must be standardised and cleaned according to the needs of the use case. This includes harmonising the time granularity (e.g., minutes, hours, days, or weeks), handling missing dates and irregular observation frequencies, resolving duplicate or delayed records, and applying a consistent calendar definition (e.g. ISO weeks, fiscal calendars, or retailer-specific week definitions). Public holidays, promotional periods, and other special events may also largely affect the acquisitions.

Many problems are somewhat shared by other machine learning problems, however, time series inherently amplify these issues due to the temporal dependency and the business logic embedded in the underlying data.

11.3 Algorithms

Here we list a series of algorithms used in time series analysis.

Moving Average

Moving average is an algorithm used to smooth a curve according to a moving window, defined as:

$$M_k(x_n) = \frac{1}{k} \sum_{i=n-k+1}^n x_i$$

where k is the window size. The first $k - 1$ points cannot be computed without padding the series or using a smaller window. A larger value of k produces a smoother curve, but can hide short-term variations. Normally, k is an odd number below 10, but the optimal value depends on the frequency and noise level of the acquired data.

Exponential Smoothing

Exponential smoothing forecasts the next value as a weighted average of the current observation and the previous forecast, giving more weight to recent data. The simple exponential smoothing procedure is:

$$\hat{x}_{t+1} = \alpha x_t + (1 - \alpha)\hat{x}_t = \alpha x_t + \alpha(1 - \alpha)x_{t-1} + \alpha(1 - \alpha)^2 x_{t-2} + \dots,$$

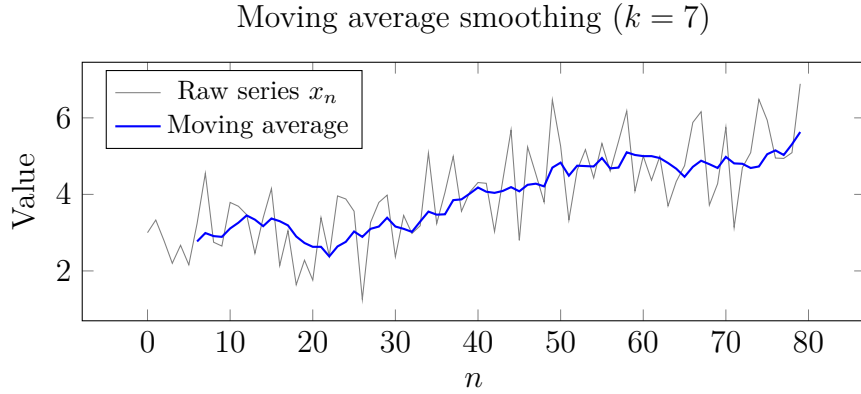


Figure 4: Moving average smoothing of a noisy series for $k = 7$.

where $\alpha \in [0, 1]$ is the smoothing parameter. A high α makes the forecast react quickly to recent changes; a low α produces a smoother, slower-reacting estimate. This technique is used similarly to the moving average, but it serves different contexts. For instance, if we have a large change at some point of our curve, the moving average would lose that information as soon as we leave its window. Also, it is easy to update, since we only need the current observation and the previous smoothed one. This basic form assumes no trend or seasonality. Holt's method extends it with a trend term, and Holt-Winters' method adds seasonality on top of that.

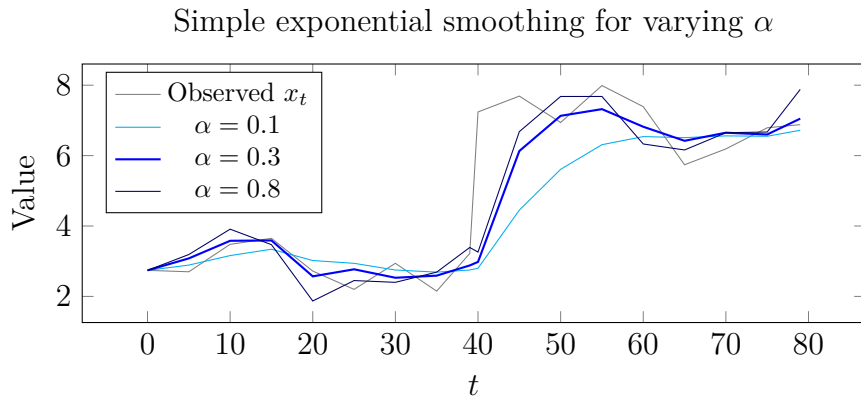


Figure 5: Exponential smoothing for varying α , reacting differently to a level shift near $t = 40$.

Holt-Winters Method

The Holt-Winters method extends exponential smoothing by tracking three evolving components: level l_t , trend b_t , and seasonal effect s_t . Each is updated recursively as new observations arrive, and the h -step-ahead forecast (additive form) is:

$$\hat{x}_{t+h} = l_t + h b_t + s_{t+h-m},$$

where m is the seasonal period. In particular, the one-step-ahead forecast is $\hat{x}_{t+1} = l_t + b_t + s_{t+1-m}$.

This additive form is appropriate when seasonal fluctuations have roughly constant magnitude. A multiplicative version (replacing sums with products for the seasonal term) is used instead when the size of the seasonal swings grows with the level of the series.

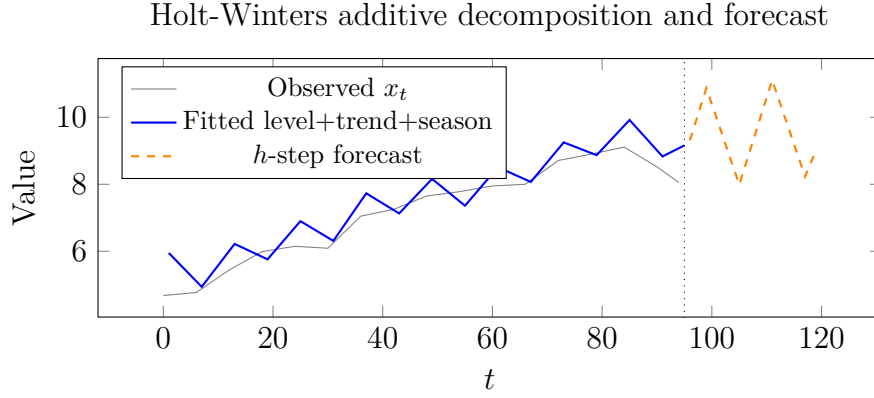


Figure 6: Holt-Winters additive fit and forecast for a trending, seasonal series ($m = 12$).

ARIMA

ARIMA (AutoRegressive Integrated Moving Average) is a model family for forecasting a single time series from its own past values. It is written as $\text{ARIMA}(p, d, q)$, combining three ingredients:

- **AR**(p): the value depends on its own previous p values.
- **I**(d): the series is differenced d times to remove trends and make it stationary (constant mean/variance over time).
- **MA**(q): the value depends on the previous q forecast errors (not a moving-average smoother).

One important concept to introduce is *differencing*. If a trend is present, we work with differences instead of raw values:

$$\nabla x_t = x_t - x_{t-1}.$$

d is the number of times this is applied before the series looks stationary (usually $d = 0$ or 1). Now we discuss the AR term. For this one, past *values* predict the current one:

$$x_t = c + \phi_1 x_{t-1} + \cdots + \phi_p x_{t-p} + \epsilon_t.$$

For the MA term instead, past *forecast errors* predict the current one:

$$x_t = c + \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}.$$

If we combine the models, using the lag operator L ($Lx_t = x_{t-1}$), we get that $\text{ARIMA}(p, d, q)$ is:

$$\phi(L)(1 - L)^d x_t = c + \theta(L)\epsilon_t,$$

with

$$\phi(L) = 1 - \phi_1 L - \cdots - \phi_p L^p$$

and

$$\theta(L) = 1 + \theta_1 L + \cdots + \theta_q L^q.$$

The error ϵ_t is white noise (zero mean, constant variance, uncorrelated); a well-fitted model leaves residuals that look like white noise. Orders p, q are typically chosen using the ACF/PACF plots or AIC/BIC.

Solving the ARIMA model for x_{t+1} and for $d = 1$, we can get the next point prediction forecast for ARIMA($p, 1, q$) as:

$$x_{t+1} = c + x_t + \sum_{i=1}^p \phi_i(x_{t+1-i} - x_{t-i}) + \sum_{j=1}^q \theta_j \epsilon_{t+1-j} + \epsilon_{t+1}.$$

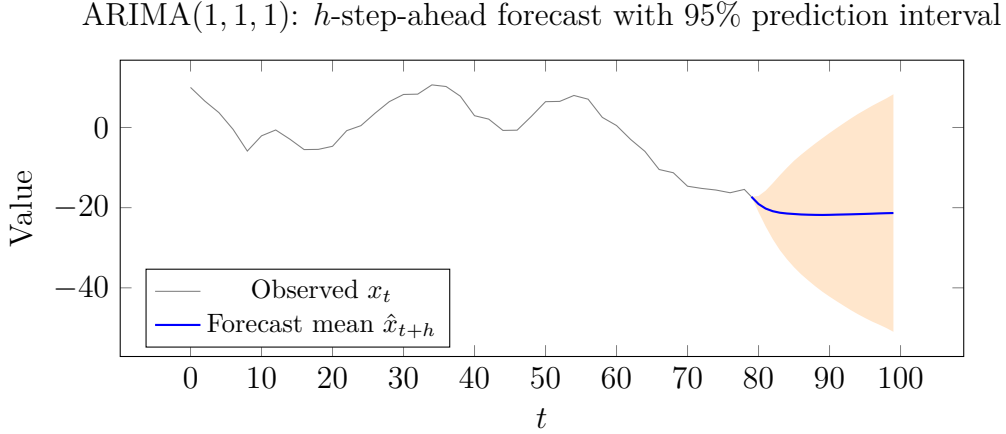


Figure 7: ARIMA(1, 1, 1) multi-step forecast ($h = 20$) continuing from the last observed point at $t = 79$, with 95% prediction interval widening over the horizon as uncertainty accumulates.

SARIMA

SARIMA (Seasonal ARIMA) extends ARIMA by adding a second, seasonal set of AR, I, and MA terms on top of the non-seasonal ones. It is written as:

$$\text{SARIMA}(p, d, q)(P, D, Q)_s,$$

where (p, d, q) are the same non-seasonal orders as in ARIMA, (P, D, Q) are their seasonal counterparts, and s is the seasonal period (e.g. $s = 12$ for monthly data with yearly seasonality).

The idea is to difference and model not only consecutive observations, but also observations one full season apart. The seasonal difference operator is

$$\nabla_s x_t = x_t - x_{t-s},$$

and the full model combines both lag operators:

$$\phi(L)\Phi(L^s)(1-L)^d(1-L^s)^D x_t = c + \theta(L)\Theta(L^s)\epsilon_t,$$

where $\Phi(L^s)$ and $\Theta(L^s)$ are the seasonal AR and MA polynomials, built the same way as $\phi(L)$ and $\theta(L)$ but using lags $s, 2s, \dots$. For example, $\Phi(L^s) = 1 - \Phi_1 L^s - \dots - \Phi_P L^{Ps}$.

SARIMA is used when a series shows both a trend and a repeated seasonal pattern, such as weekly sales cycles or yearly demand variation, that plain ARIMA cannot capture.

STL Decomposition

STL (Seasonal-Trend decomposition using Loess) splits a time series into three additive components:

$$x_t = T_t + S_t + R_t,$$

where T_t is a slowly-varying trend, S_t is a seasonal pattern that is allowed to change gradually over time, and R_t is the leftover residual. Both T_t and S_t are estimated using Loess, a locally weighted regression that fits smooth curves to local subsets of the data.

STL is a decomposition tool rather than a forecasting model: it does not predict x_{t+1} on its own, but it is commonly used for exploratory analysis, and its trend/seasonal components are often fed into a separate forecasting method (e.g. forecasting T_t and S_t separately, then recombining). It is also robust to outliers, unlike simpler decomposition methods.

Autocorrelation Analysis

Autocorrelation measures how similar a series is to a delayed (lagged) copy of itself. The autocorrelation at lag k is:

$$\rho_k = \frac{\sum_{t=k+1}^n (x_t - \bar{x})(x_{t-k} - \bar{x})}{\sum_{t=1}^n (x_t - \bar{x})^2}.$$

A value of ρ_k close to 1 (or -1) means observations k steps apart tend to move together (or oppositely); a value near 0 means little linear relationship at that lag.

This is a diagnostic tool rather than a forecasting model: the autocorrelation function (ACF) and partial autocorrelation function (PACF) are used together to spot periodic patterns and to choose the orders p and q of an ARIMA model.

Fourier Analysis

Fourier analysis represents a time series as a sum of sine and cosine waves of different frequencies:

$$x_t = a_0 + \sum_{k=1}^K \left[a_k \cos\left(\frac{2\pi kt}{n}\right) + b_k \sin\left(\frac{2\pi kt}{n}\right) \right].$$

Each term k captures a periodic pattern that repeats k times over the length- n series; the coefficients a_k, b_k give the strength of that pattern.

This decomposition reveals periodicities that may not be obvious by looking at the raw series over time. In practice, the dominant frequencies can be used as seasonal predictors in a regression, or the series can be forecast by extrapolating the fitted sinusoids forward: $\hat{x}_{t+1}$ is obtained by evaluating the fitted sum at $t + 1$.

Change Point Detection

Change point detection locates times at which the statistical behaviour of a series shifts, whether in mean, variance, trend, or seasonality. A simple test compares the average level before and after a candidate point τ :

$$\Delta_\tau = \left| \frac{1}{\tau} \sum_{t=1}^{\tau} x_t - \frac{1}{n - \tau} \sum_{t=\tau+1}^n x_t \right|,$$

where τ is the position number of the considered point. A large Δ_τ suggests a structural break at τ . In More sophisticated methods test many candidate points and/or other statistics (variance, slope) rather than just the mean.

This is a detection tool, not a forecasting model: it is used to monitor processes, flag regime changes in markets, and identify when external events have shifted the series.

Anomaly Detection

Anomaly detection flags observations that deviate sharply from expected behaviour. The standard approach compares each point to the value predicted by a forecasting or decomposition model, giving a residual:

$$r_t = x_t - \hat{x}_t.$$

An observation is flagged as anomalous when its residual is unusually large relative to the typical residual size:

$$|r_t| > \lambda \sigma_r,$$

where σ_r is the standard deviation of the residuals and λ is a threshold (commonly 2 or 3).

Because this relies on residuals rather than raw values, it can be layered on top of any of the models above (ARIMA, Holt-Winters, STL, etc.). It is used to detect unexpected demand, sensor failures, operational incidents, and unusual financial transactions.

12 Likelihood Function

The Likelihood function is a fundamental concept in statistics that flips the perspective of probability of a random variable given some parameters, to the probability of having some parameters given some specific observations of the random variable.

12.1 Probability Functions

A probability function of a generic process describes how likely is to observe any event among the possible ones. It's crucial to distinguish between *discrete* and *continuous* probability functions.

A **discrete probability function** describes a process that can assume only a finite or countable number of values. The probability function is called **probability mass function** and it is defined as:

$$P_\chi(x) = P(\chi = x) \tag{9}$$

so it is the probability for the random variable χ to assume exactly the value x . It must satisfy the following properties:

- $P_\chi(x) \geq 0 \ \forall x$
- $\sum_x P_\chi(x) = 1$
- $P(A) = \sum_{x \in A} P_\chi(x)$

For a **continuous probability function** instead it doesn't make sense to ask how much likely is to observe a specific value, since we are dealing with a continuous random variable. For this reason we define the **probability density function** as:

$$\rho_\chi(x) : P(a \leq \chi \leq b) = \int_a^b \rho_\chi(x) dx \quad (10)$$

where $P(\chi)$ is the cumulative distribution function. Again we require:

- $\rho_\chi(x) \geq 0 \forall x$
- $\int_{-\infty}^{+\infty} \rho_\chi(x) dx = 1$
- $P(a \leq \chi \leq b) = \int_a^b \rho_\chi(x) dx$

12.2 Maximum Likelihood Estimation

Given some random variable $\chi \sim f_\chi(x|\vec{\theta})$ where $\vec{\theta}$ are eventual fixed parameters, assuming the observations are independent and identically distributed we define the **likelihood** as:

$$\mathcal{L}(\vec{\theta}|\vec{x}) = \prod_{i=1}^N f_\chi(x_i|\vec{\theta}) , \quad (11)$$

which corresponds to the joint probability density (or mass, for discrete variables) of observed data $\vec{x} = \{x_1, x_2, \dots, x_N\}$ as a function of the parameters $\vec{\theta}$. Now this definition makes perfect sense for *discrete* random variables, but also for *continuous* ones, since we can consider the joint probability densities instead of the probabilities themselves.

We defined $\mathcal{L}(\vec{\theta}|\vec{x})$ so that if we maximize it around the observed data $\vec{x}$, we would get the parameters $\hat{\vec{\theta}}$ that better explain the data. This method is called **Maximum Likelihood Estimation**; for the sake of calculations usually we consider the **log-likelihood** instead:

$$\ln \mathcal{L}(\vec{\theta}|\vec{x}) = \sum_{i=1}^N \ln f_\chi(x_i|\vec{\theta}) \quad (12)$$

and we maximize it with respect to $\vec{\theta}$:

$$\begin{aligned} \hat{\vec{\theta}} &= \arg \max_{\vec{\theta}} \ln \mathcal{L}(\vec{\theta}|\vec{x}) \\ \Rightarrow \left(\frac{d\mathcal{L}(\vec{\theta}|\vec{x})}{d\vec{\theta}} \right)_{\vec{\theta}=\hat{\vec{\theta}}} &= \vec{0}. \end{aligned}$$

13 P-Value

The p-value is a fundamental tool used to quantify how much something is likely to be true. We start from a statistical assumption, called **null hypothesis** H_0 , and we want to estimate how much H_0 fits the observed data. The formal definition is "*Assuming that H_0 is true, the p-value is the probability of obtaining a test statistic as extreme as at least the one measured from the experimental data*".

$$p \equiv P(t > t_{obs} | H_0) , \quad t \sim f(t) .$$

Now we need to define H_0 and the test statistic t .

- H_0 : My data comes from the distribution $f(\vec{\theta})$, or "my data has mean in μ ", etc..
- **Test Statistics** : Quantity strictly related to the null hypothesis, and measured from the observed data. It should be tailored on the null hypothesis and on what we want to achieve.

Each test statistics has its particular scope in mind. Here we list few of them.

- **t-test** : Tests if the data has average in μ_0 . Variance unknown, small dataset.

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{N}}, \quad s^2 = \frac{\sum^N (x_i - \bar{x})^2}{N - 1}.$$

- **Pearson's Chi-squared** : Tests the goodness of fit. $\hat{x}_i$ predicted, x_i observed.

$$\chi^2 = \sum_{i=1}^N \frac{(x_i - \hat{x}_i)^2}{\hat{x}_i}.$$

- **F-test (ANOVA)** : Compare the variance between two groups. Used to see if two groups are different or not.

$$f = \frac{s_1^2}{s_2^2}.$$

In ANOVA notation, s_1^2 is the within group variance, s_2^2 is the between groups variance.

the distribution of p varies depending on if H_0 is true or not. In fact, if H_0 is true, p must be distributed uniformly:

- The test statistics follows a distribution $f(t)$
- Sampling from any cumulative distribution means that any percentile has the same probability to occur, since it is the interval $[0, 1]$.
- If we define the cumulative of t as $F_{H_0}(T \leq t|H_0) = P(t)$, then $p \sim P(t)$ must be uniform.

14 Monte Carlo Simulation

15 Machine Learning Techniques

This chapter is only aimed to introduce the most relevant techniques in the ML world. We will also briefly view the architectures of models like simple neural network and transformers with memory.

15.1 K-Mean Clustering

This clustering technique is a straightforward and extremely efficient technique to identify clusters in a dataset, given a metric. The idea is simple:

1. Define the number of clusters k .
2. Randomly assign the centroid position of each cluster.
3. Evaluate the distance of each point p_i to all the centroids $\{\mu_j^t\}_0^k$, and assign the point to the closest centroid. This will produce the set of points in a cluster j as $S_j^t = \{p : \|p - \mu_j\| \leq \|p - \mu_i\|, \forall j\}$
4. Calculate the new centroid of each cluster according to the new points assigned.
$$\mu_j^{t+1} = \frac{1}{|S_j^t|} \sum_{p_i \in S_j^t} p_i$$
5. Repeat from step 3.

It's clear that the algorithm as expressed above is poorly efficient, and there are many different implementations that speeds up the process, like Minibatch K means.

15.2 Principal Component Analysis (PCA)

PCA is a dimensionality reduction technique that is based on projecting the data into the orthogonal eigenvectors associated to the largest eigenvalues. The aim is to find the directions that still carries most of the variance of the data, so that little information is lost.

Let $X \in \mathbb{R}^{n \times m}$ be the $n \times m$ matrix data, n points and m features. We want to standardize the data, so having all the columns (features) with zero mean and rescaled by the feature variance so that they are comparable:

$$Z_{i,j} = \frac{X_{i,j} - \bar{X}_j}{\sigma_j}.$$

Once we standardized the data, we want to find a direction in the feature space $\vec{r} \in \mathbb{R}^m$ onto which the projection of the data has the maximum variance of the samples:

$$\begin{aligned} \text{projection: } & \vec{z}_i^\top \cdot \vec{r} \\ \text{variance: } & \text{Var}[\vec{z}_i^\top \cdot \vec{r}] = \frac{1}{n-1} \sum_{i=1}^n (\vec{z}_i^\top \cdot \vec{r})^2 \\ \rightarrow & \text{Var}[Z\vec{r}] = \frac{1}{n-1} \vec{r}^\top Z^\top Z \vec{r} \equiv \vec{r}^\top S \vec{r} \end{aligned}$$

with S being the covariance matrix $S = \frac{1}{n-1} Z^\top Z$. Now, we want to maximize $\text{Var}[Z\vec{r}]$ over a specific direction $\vec{r}$ with the constrain of orthogonality $\vec{r}^\top \vec{r} = 1$. For this task, we use Lagrange multipliers:

$$\begin{aligned} L(\vec{r}, \lambda) &= \vec{r}^\top S \vec{r} - \lambda(\vec{r}^\top \vec{r} - 1) \\ \rightarrow \nabla_{\vec{r}} L &= 2S\vec{r} - 2\lambda\vec{r} = 0 \end{aligned}$$

and we can actually see that this is the eigenvalue problem of the covariance matrix of the data:

$$S\vec{r} = \lambda\vec{r}.$$

Therefore the variance captured on the specific directions is given by the eigenvalue, and we usually need only the first few to embed most of the variance.

15.3 Gradient Descent

The gradient descent is an iterative optimization technique used to minimize a function, in our cases a loss function. The name describes exactly the idea behind: consider the gradient of a function, and go in the opposite direction.

Let's consider a dataset with N samples $\{\vec{x}_i, y_i\}_1^n$, having $\vec{x}_i \in \mathbb{R}^m$ the input features for the i -th result and $y_i \in Y$ defined in the target space is the result we want to predict. We develop a model $f(\vec{x}|\vec{\theta})$ parametrized with the parameters vector $\vec{\theta} \in \mathbb{R}^d$ which will produce the single prediction $y \in Y$.

In order to find the optimal parameters $\vec{\theta}$ we consider the input data fixed, and we let the parameters change $y_i = f(\vec{\theta}|\vec{x}_i)$, so this becomes an optimization problem. We define a loss function $L(f(\vec{\theta}|\vec{x}_i), y_i) : \mathbb{R}^p \times Y \rightarrow \mathbb{R}$ which quantifies how much the prediction differs from y_i . Now we can define a global cost function by considering all the points in the dataset:

$$C(\vec{\theta}) = \frac{1}{n} \sum_{i=1}^n L(f(\vec{\theta}|\vec{x}_i), y_i)$$

The best set of parameters is the one that minimizes the cost function:

$$\vec{\theta} = \arg \min_{\vec{\theta}} C(\vec{\theta}),$$

since it is a metric that shows how distant are the predictions from the real values.

At this point, we just need to find where are the minimum points of the cost function $C(\vec{\theta})$. In order to do that, we simply consider the gradient of $C(\vec{\theta})$ – which we know is orthogonal to the level curves and points to where the function increases – and we go in the opposite direction:

$$\vec{\theta}_{i+1} = \vec{\theta}_i - \eta \nabla C(\vec{\theta}_i).$$

How do we get this algorithm:

1. We start the discussion considering Taylor expansion

$$C(\vec{\theta} + \Delta\vec{\theta}) = C(\vec{\theta}) + (\nabla C(\vec{\theta}))^\top \Delta\vec{\theta} + o(\|\Delta\vec{\theta}\|^2)$$

2. Since we want to "go in the opposite direction" of the gradient, we choose

$$\Delta\vec{\theta} \propto -\nabla C(\vec{\theta})$$

3. In order to match the dimensionality of $\vec{\theta}$, we do need a dimensional parameter $\eta > 0$ so that

$$\Delta\vec{\theta} = -\eta \nabla C(\vec{\theta})$$

4. Substituting back in the Taylor expansion we get

$$C(\vec{\theta} + \Delta\vec{\theta}) \simeq C(\vec{\theta}) - \eta \|\nabla C(\vec{\theta})\|^2$$

which shows that for "small" steps we know that $C(\vec{\theta} + \Delta\vec{\theta}) \leq C(\vec{\theta})$, thus the convergence to a local minimum of $\vec{\theta}$.

A note on η parameter. Usually in ML applications it is considered adimensional, since we are interested more on the "empirical" consequences rather than dimension considerations.

However, ideally we should consider $[\eta] = \frac{[\theta]^2}{[C]}$.

In practice, in order to apply the gradient descent in ML algorithms, we do need to evaluate all the partial derivatives. Libraries like *torch* and similar does that by default.

15.4 Gradient Boosting

Gradient boosting is a supervised training technique which is based on the use of an initial model, then that model's output is improved by the contribution of other weak models trained sequentially on the residuals.

Given some data $\vec{x}$ and the target $\vec{y}$, we want to find a function $F(\vec{x})$ that minimize the loss function:

$$\sum_{i=1}^N L(y_i - F(x_i))$$

so that we find $F(\vec{x})$ iteratively. Now, we first start by training a generic model – usually a tree – $F_0(\vec{x})$ and we calculate the residuals for that model:

$$\varepsilon_{i,m} = \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}} .$$

Each residual $\vec{\varepsilon}_m$ is calculated with the previous model F_{m-1} , and it is used as target to train a new weak model $f_m(\vec{x})$, from $(\vec{x}, \vec{\varepsilon}_m)$. This model will contribute to the m -th step as:

$$F_m(\vec{x}) = F_{m-1} + \eta \gamma_m f_m(\vec{x}) ,$$

with η being the learning rate and γ_m being the optimal scaling multiplier:

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, F_{m-1}(x_i) + \gamma f_{m-1}(\vec{x})) .$$

In the end our final model function $F(\vec{x})$ will be the sequence of all the weak models contribution on the error of the previous output:

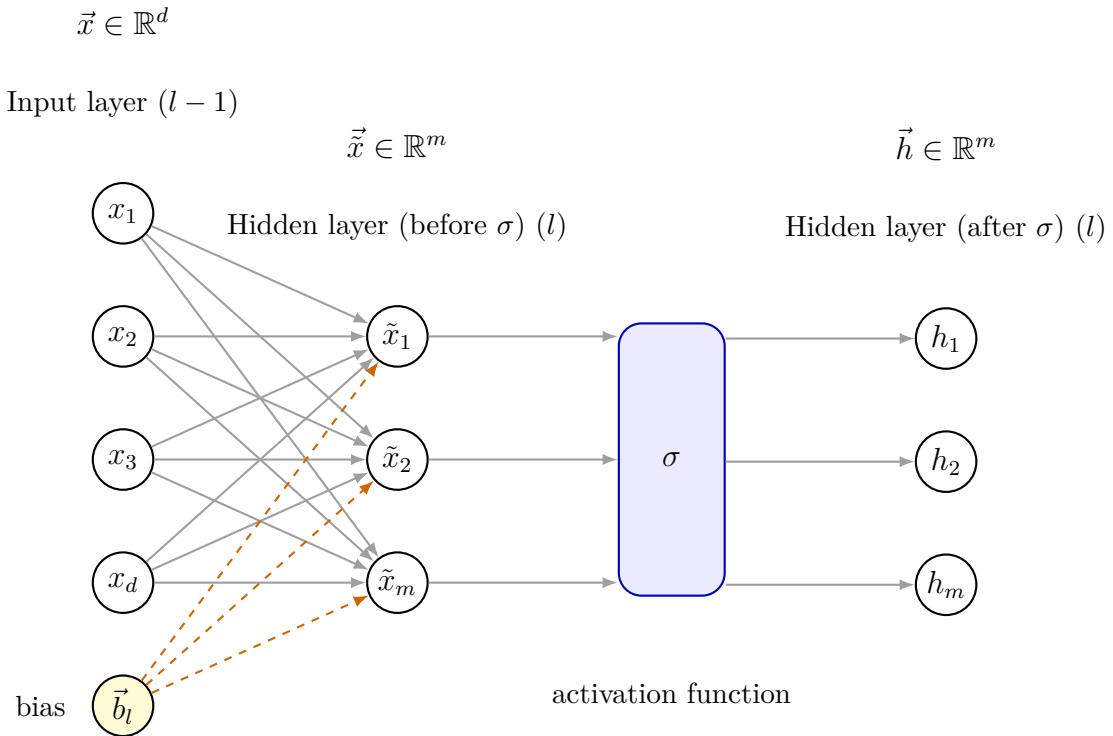
$$F(\vec{x}) = \sum_{m=1}^M F_m(\vec{x}) .$$

Usually, f_m is a small tree, that can be both used as regressor and classifier, depending on the context.

15.5 Simple Neural Networks

This section will only be a very quick description of the basic components of neural networks. Any neural network is a structure that processes an input $\vec{x} \in \mathbb{R}^d$ and returns an output $\vec{y} \in \mathbb{R}^p$.

The input is processed by “layers”, which apply parameterized affine transformations, and then the output is modified by a non-linear function called an activation function. Each value in a layer is called a “node” or neuron; nodes are not themselves parameters of the neural network. In general, each node contains n weights and one bias: for instance, in a fully connected network, every node contains as many weights as the dimension of its input vector, plus one bias. Conventionally, W denotes the weight tensor, where for each node we have a vector of weights, and b denotes the collection of bias vectors, containing one bias vector per layer.



One very important thing that we also need to introduce before going through each neural network architecture is the backpropagation algorithm. Since any NN is a collection of functions applied sequentially, and since we need at some point to evaluate the gradient, one particularly useful technique to speed up the process is backpropagation.

In practice, once we have defined the cost function $C(W)$, where

$$W = (w_1, w_2, \dots, w_N)$$

denotes the collection of all the parameters of the network (weights and biases), we

eventually need to compute the gradient:

$$\nabla C(W) = \begin{bmatrix} \frac{\partial C(W)}{\partial w_1} \\ \frac{\partial C(W)}{\partial w_2} \\ \vdots \\ \frac{\partial C(W)}{\partial w_N} \end{bmatrix}.$$

Each derivative is a partial one because C is a function of all N parameters at once: every component of $\nabla C(W)$ is taken with respect to a single w_j , holding the remaining parameters fixed.

Since our cost function is in the form

$$C(W) = \frac{1}{n} \sum_{i=1}^n L(f(\vec{h}_i | W), y_i),$$

we need to apply the chain rule through all the layers. Backpropagation is simply an efficient way of doing this: the values are computed from the input to the output, while the derivatives are computed in the opposite direction, starting from the output layer.

Let $\tilde{x}_i^{(l)}$ be the value of the i -th node in layer l before the activation function, and let

$$h_i^{(l)} = \sigma(\tilde{x}_i^{(l)})$$

be its output. In what follows we write C for $C(W)$ to lighten the notation. For a single weight $w_{ik}^{(l)}$, connecting node k in layer $l-1$ to node i in layer l , the gradient is:

$$\frac{\partial C}{\partial w_{ik}^{(l)}} = \frac{\partial C}{\partial \tilde{x}_i^{(l)}} \frac{\partial \tilde{x}_i^{(l)}}{\partial w_{ik}^{(l)}}.$$

The value before the activation function is defined as:

$$\tilde{x}_i^{(l)} = \sum_{r=1}^{n_{l-1}} w_{ir}^{(l)} h_r^{(l-1)} + b_i^{(l)},$$

where n_{l-1} is the number of nodes in layer $l-1$. By differentiating this expression with respect to $w_{ik}^{(l)}$, we obtain:

$$\frac{\partial \tilde{x}_i^{(l)}}{\partial w_{ik}^{(l)}} = h_k^{(l-1)}.$$

Therefore:

$$\frac{\partial C}{\partial w_{ik}^{(l)}} = \frac{\partial C}{\partial \tilde{x}_i^{(l)}} h_k^{(l-1)}.$$

Similarly, for the bias $b_i^{(l)}$, we have:

$$\frac{\partial C}{\partial b_i^{(l)}} = \frac{\partial C}{\partial \tilde{x}_i^{(l)}}.$$

The value $h_k^{(l-1)}$ is already known from the forward computation, while $\frac{\partial C}{\partial \tilde{x}_i^{(l)}}$ – also called local gradient – is calculated by applying the chain rule, starting from the cost function and proceeding from the output layer towards the previous layers. This reverse evaluation of the derivatives is what is called backpropagation. Once $\frac{\partial C}{\partial \tilde{x}_i^{(l)}}$ is known, the gradients are:

$$\frac{\partial C}{\partial w_{ik}^{(l)}} = \frac{\partial C}{\partial \tilde{x}_i^{(l)}} h_k^{(l-1)}$$

$$\frac{\partial C}{\partial b_i^{(l)}} = \frac{\partial C}{\partial \tilde{x}_i^{(l)}}.$$

These gradients tell us how each parameter affects the cost function. Given a learning rate $\eta > 0$, gradient descent updates each parameter $\theta \in W$ as:

$$\theta \leftarrow \theta - \eta \frac{\partial C}{\partial \theta}.$$

The negative sign moves the parameters in the direction in which the cost function decreases. By repeating the forward computation, backpropagation, and the parameter update, the neural network progressively reduces its error on the training data.

Feed Forward NN

Let $\vec{x} \in \mathbb{R}^d$ be an input feature vector and let the network contains M layers. Each layer l contains $W^{(l)}$ parameters and a vector bias $\vec{b}_l$, with $W^{(l)}$ being the weight matrix of the l -th layer – could be a vector or matrix. The network function is

$$f : \mathbb{R}^d \rightarrow \mathbb{R}^p$$

$$\vec{x} \mapsto f(\vec{x} | W) = \vec{y}$$

and For each layer $l = 0, \dots, M - 1$ the network computes:

$$\vec{h}_{l+1} = \sigma_l(W^{(l)}\vec{h}_l + \vec{b}_l),$$

with $W^{(l)} \in \mathbb{R}^{n_{l+1} \times n_l}$ where n_{l+1} is the dimension of the next layer, n_l is the dimension of the incoming input. σ_l is the activation function for the l -th layer and it is applied component wise. $\vec{b}_l$ is the bias for the l -th layer.

In the end, the process is:

1. The initial input $\vec{x} = \vec{h}_0$
2. $\vec{h}_1 = \sigma_0(W^{(0)}\vec{h}_0 + \vec{b}_0)$ and so on ...
3. The final step will produce the output $\vec{y} = \vec{h}_M$

Convolutional NN

Let $X \in \mathbb{R}^{H \times W \times C}$ be an input tensor (e.g. an image) with height H , width W and C input channels, and let the network contain M layers. Each layer l contains a set of learnable kernels $K^{(l)}$ and a bias vector $\vec{b}_l$, with $K^{(l)}$ being the collection of filters of the l -th layer.

The network function is

$$f : \mathbb{R}^{H \times W \times C} \rightarrow \mathbb{R}^p$$

$$X \mapsto f(X | K) = \vec{y}$$

and for each layer $l = 0, \dots, M - 1$ the network computes:

$$H_{l+1}[:, :, k] = \sigma_l \left(\left(K_k^{(l)} * H_l \right) + b_{l,k} \right), \quad k = 1, \dots, C_{l+1},$$

with $K_k^{(l)} \in \mathbb{R}^{k_h \times k_w \times C_l}$ the k -th kernel of the l -th layer, C_l the number of channels of the incoming feature map, C_{l+1} the number of output channels (i.e. the number of kernels), and $*$ denoting the (cross-correlation) convolution operator with a given stride and padding. σ_l is the activation function for the l -th layer, applied component wise, and $b_{l,k}$ is the scalar bias associated with the k -th output channel.

Layers may be interleaved with pooling operations P_l (e.g. average-pooling) that reduce the spatial dimensions:

$$H_{l+1} = P_l(H_{l+1}).$$

In the end, the process is:

1. The initial input $X = H_0$
2. $H_1 = \sigma_0(K^{(0)} * H_0 + \vec{b}_0)$, optionally followed by pooling, and so on ...
3. The final feature map H_M is flattened into a vector and passed through one or more fully connected layers (as in the Feed Forward NN) to produce the output $\vec{y}$

Recurrent NN

Let $\vec{x}_t \in \mathbb{R}^d$, $t = 1, \dots, T$, be a sequence of input feature vectors, and let $\vec{h}_t \in \mathbb{R}^n$ denote the hidden state of the network at time step t . The network shares the same parameters W_{hh} , W_{xh} , W_{hy} and biases $\vec{b}_h$, $\vec{b}_y$ across all time steps.

The network function is

$$f : \mathbb{R}^d \times \mathbb{R}^n \rightarrow \mathbb{R}^n \times \mathbb{R}^p$$

$$(\vec{x}_t, \vec{h}_{t-1}) \mapsto f(\vec{x}_t, \vec{h}_{t-1} | W) = (\vec{h}_t, \vec{y}_t)$$

and for each time step $t = 1, \dots, T$ the network computes:

$$\vec{h}_t = \sigma_h(W_{hh} \vec{h}_{t-1} + W_{xh} \vec{x}_t + \vec{b}_h), \quad \vec{y}_t = \sigma_y(W_{hy} \vec{h}_t + \vec{b}_y),$$

with $W_{hh} \in \mathbb{R}^{n \times n}$ the recurrent weight matrix, $W_{xh} \in \mathbb{R}^{n \times d}$ the input-to-hidden weight matrix, and $W_{hy} \in \mathbb{R}^{p \times n}$ the hidden-to-output weight matrix. σ_h and σ_y are the activation functions applied component wise, and $\vec{b}_h$, $\vec{b}_y$ are the corresponding bias vectors. In the end, the process is:

1. The initial hidden state $\vec{h}_0$ is set (typically to $\vec{0}$)
2. $\vec{h}_1 = \sigma_h(W_{hh}\vec{h}_0 + W_{xh}\vec{x}_1 + \vec{b}_h)$ and so on ...
3. At each step t the output $\vec{y}_t = \sigma_y(W_{hy}\vec{h}_t + \vec{b}_y)$ is produced, and the final hidden state $\vec{h}_T$ summarizes the whole input sequence

15.6 Reinforcement Learning

This topic requires a section of its own, due to its complexity and variety of applications. Also, the results of applying reinforcement learning can be outstanding, granted that we have enough computational power and a well-defined problem.

RL is based on the concept of an *agent* placed in an environment with a task to complete, and a *reward function* that gives the agent feedback in terms of quantitative rewards. The situations that are best suited for RL are those that can be well defined in terms of states, actions and outcomes, like finding the best path in a 3D environment or to win a chess match.

The mathematical framework for RL is based on a *Markov Decision Process* (MDP), represented by the tuple (S, A, P, R, γ) . S is the set of information about the system given to the agent – terrain closest coordinates, pieces disposition on the board

$$S = \{s_i\}_{i=1}^N \longrightarrow \left\{ \begin{array}{l} \text{all possible arrangements of pieces on a board,} \\ \text{including configurations with pieces removed} \end{array} \right\}.$$

A is the set of actions the agent can perform

$$A = \{a_i\}_{i=1}^M \rightarrow \{\text{left, right, up, down}\},$$

P is the transition probability between states

$$P(u | s, a) = \Pr(S_{t+1} = u | S_t = s, A_t = a),$$

R is the reward associated with a transition or a state-action pair

$$R(s, a, u) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a, S_{t+1} = u],$$

and $\gamma \in [0, 1]$ is the factor weighting future rewards. The behavior of the agent in the environment is described by the *policy* π , which describes the probability that the agent will choose a specific action given the current state

$$\pi(a | s) = \Pr(A_t = a | S_t = s).$$

It is important to notice that the *reward* R is an expected value, since it can be deterministic or stochastic. Imagine an agent aiming to minimize the time to complete a race track, it will receive a deterministic reward – the lower the time, the better the reward – while a more natural environment could give rewards based on probabilities – probability to get ill, or whatever.

At each time step t , the agent observes a state S_t , selects an action A_t according to a policy π , receives a reward R_{t+1} , and enters a new state S_{t+1} . The agent's objective is to

maximize the expected total reward:

$$G_t = \sum_{k=0}^{+\infty} \gamma^k R_{t+k+1} \Rightarrow \text{Continuing task}$$

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \Rightarrow \text{Episodic task ,}$$

where a *continuing task* is a task that does not have an end – i.e. continuing monitoring a value – while the *episodic task* is something that eventually will end – i.e. a racing track game.

The final important quantities we have to introduce are the *value functions*:

$$V_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

$$Q_\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a] ,$$

in particular expressed recursively through *Bellman equations*:

$$V_\pi(s) = \sum_a \pi(a \mid s) \sum_{u,r} p(u, r \mid s, a) [r + \gamma V_\pi(u)]$$

$$Q_\pi(s, a) = \sum_{u,r} p(u, r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid u) Q_\pi(u, a') \right] .$$

The function $Q_\pi(s, a)$ tells which is the expected value given a policy π , a state s and an action a , which is used to measure the quality of the agent action, and $V_\pi(s)$ tells us "how good is" to be in a state s with a policy π and it's used to measure the quality of the policy in every state.

We want to maximize the functions, in particular Q_π since it's responsible for the actions of the agent, meaning that:

$$Q^*(s, a) = \max_{\pi} Q_\pi(s, a)$$

is the maximum expected return obtainable when the agent starts in a state s , selects action a and then behaves optimally. Once $Q^*(s, a)$ is known, the best action for a state is found as:

$$a^*(s) = \arg \max_{a \in A(s)} Q^*(s, a) ,$$

so we maximize over actions now instead of policies as before. Equivalently, the optimal deterministic policy assigns probability one to this action.

Note that we wrote down the equations considering generic stochastic setups, but for deterministic tasks we would remove all the uncertainties related to state transitions; however, during training we usually introduce some exploration in order to consider different actions, even the ones that would not normally be selected.

In practice, finding Q^* exactly can be computationally impossible for large problems. Therefore, the agent learns an approximation Q and selects the next action using an ε -greedy policy. Assuming a unique greedy action, this policy is defined as:

$$\pi_\varepsilon(a \mid s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|A(s)|}, & a = \arg \max_{b \in A(s)} Q(s, b), \\ \frac{\varepsilon}{|A(s)|}, & \text{otherwise.} \end{cases}$$

The term $\varepsilon/|A(s)|$ appears because, during random exploration, every action can be selected, including the greedy one.

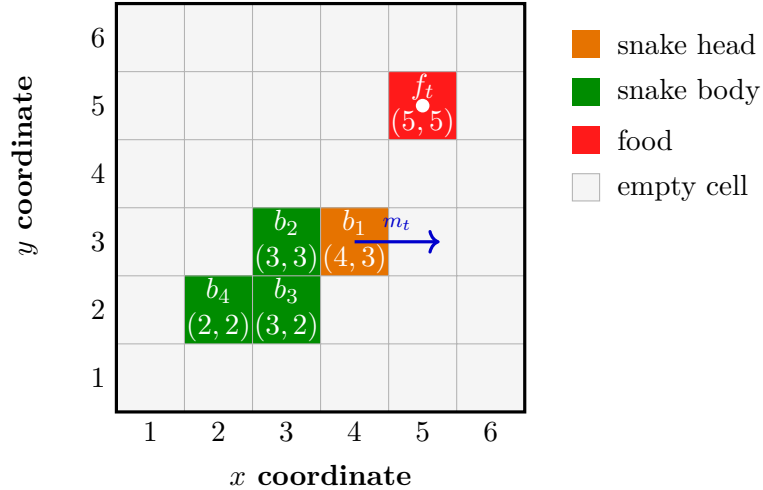
By substituting this policy into the Bellman equation for Q_π , we obtain:

$$\begin{aligned} Q_{\pi_\varepsilon}(s, a) &= \sum_{u, r} p(u, r \mid s, a) \left[r + \gamma \sum_{a'} \pi_\varepsilon(a' \mid u) Q_{\pi_\varepsilon}(u, a') \right] \\ &= \sum_{u, r} p(u, r \mid s, a) \left[r + \gamma \left((1 - \varepsilon) \max_{a'} Q_{\pi_\varepsilon}(u, a') + \frac{\varepsilon}{|A(u)|} \sum_{a'} Q_{\pi_\varepsilon}(u, a') \right) \right]. \end{aligned}$$

This is the Bellman equation evaluated using the ε -greedy policy: with probability $1 - \varepsilon$ the agent follows the best currently known action, while with probability ε it selects an action randomly. During training, ε is usually reduced gradually, while after training we can set $\varepsilon = 0$ to always select the best learned action.

Snake example

We want to create an agent trained to win a 2D snake game. The items making the game are the following: *snake's head*, *snake's body*, *food*, *empty grid cells* and *grid boundaries*. One episode starts at the time zero and ends when snake hits the boundary or its own body. We define the possible actions $A = \{\text{up, down, left, right}\}$ and we define the current



state – meaning the set of information available to the agent – as

$$S_t = \{c_i = (x_i^{(c)}, y_i^{(c)})\}_{i=1}^{N-M_t-1} \cup \{b_j = (x_j^{(b)}, y_j^{(b)})\}_{j=1}^{M_t} \cup \{(f_x, f_y)\}$$

, where c_i is the position of the i -th empty cell, b_j is the position of the j -th part of the snake body and f_x, f_y are the x and y positions of the food. Therefore, we will always have N elements in the set.

The smarter approach to the problem is however to consider as actions the only $A = \{\text{continue straight, turn left, turn right}\}$, and for the states only *closest dangers* (d), *current movement* m and *direction to food* f as states, meaning

$$S_t = \{d_u, d_d, d_l, d_r, m_u, m_d, m_l, m_r, f_u, f_d, f_l, f_r\}$$

being all 0, 1 values. This approach is faster, but it won't describe the totality of the setup, since we lack the specific coordinates. The reward function can be defined as

follows:

$$R_{t+1} = \begin{cases} +10, & \text{if hits food} \\ -10, & \text{if hits danger} \\ -0.1, & \text{if hits empty cell} \end{cases}$$

The agent selects its actions according to an ε -greedy policy. Since the action set contains three possible actions, assuming a unique greedy action, the policy is:

$$\pi_\varepsilon(a \mid s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{3}, & a = \arg \max_{b \in A} Q(s, b), \\ \frac{\varepsilon}{3}, & \text{otherwise,} \end{cases}$$

where

$$A = \{\text{continue straight, turn left, turn right}\}.$$

Therefore, the Bellman equation evaluated using the ε -greedy policy becomes:

$$\begin{aligned} Q_{\pi_\varepsilon}(s, a) &= \sum_u P(u \mid s, a) \left[R(s, a, u) + \gamma \sum_{a' \in A} \pi_\varepsilon(a' \mid u) Q_{\pi_\varepsilon}(u, a') \right] \\ &= \sum_u P(u \mid s, a) \left[R(s, a, u) + \gamma \left((1 - \varepsilon) \max_{a' \in A} Q_{\pi_\varepsilon}(u, a') + \frac{\varepsilon}{3} \sum_{a' \in A} Q_{\pi_\varepsilon}(u, a') \right) \right]. \end{aligned}$$

Writing the three possible actions explicitly, we obtain:

$$\begin{aligned} Q_{\pi_\varepsilon}(s, a) &= \sum_u P(u \mid s, a) \left[R(s, a, u) + \gamma \left((1 - \varepsilon) \max \{Q_{\pi_\varepsilon}(u, \text{straight}), Q_{\pi_\varepsilon}(u, \text{left}), Q_{\pi_\varepsilon}(u, \text{right})\} \right. \right. \\ &\quad \left. \left. + \frac{\varepsilon}{3} [Q_{\pi_\varepsilon}(u, \text{straight}) + Q_{\pi_\varepsilon}(u, \text{left}) + Q_{\pi_\varepsilon}(u, \text{right})] \right) \right]. \end{aligned}$$

If the transition of the game is deterministic, the action a in state s produces only one next state u , and the equation simplifies to:

$$\begin{aligned} Q_{\pi_\varepsilon}(s, a) &= R(s, a, u) + \gamma \left((1 - \varepsilon) \max \{Q_{\pi_\varepsilon}(u, \text{straight}), Q_{\pi_\varepsilon}(u, \text{left}), Q_{\pi_\varepsilon}(u, \text{right})\} \right. \\ &\quad \left. + \frac{\varepsilon}{3} [Q_{\pi_\varepsilon}(u, \text{straight}) + Q_{\pi_\varepsilon}(u, \text{left}) + Q_{\pi_\varepsilon}(u, \text{right})] \right), \end{aligned}$$

where

$$R(s, a, u) = \begin{cases} +10, & \text{if the snake reaches the food,} \\ -10, & \text{if the snake hits a danger,} \\ -0.1, & \text{if the snake reaches an empty cell.} \end{cases}$$

When u is a terminal state, meaning that the snake has hit a danger, there are no future rewards and the equation becomes:

$$Q_{\pi_\varepsilon}(s, a) = -10 .$$

Initially, the agent does not know the quality of the different actions, so we initialize:

$$Q(s, a) = 0 \quad \forall s \in S, a \in A .$$

At each time step t , the agent observes the current state S_t , selects an action A_t using the ε -greedy policy, receives the reward R_{t+1} and observes the next state S_{t+1} . The value associated with the state-action pair (S_t, A_t) is then updated as:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_{a' \in A} \pi_\varepsilon(a' | S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t) \right] ,$$

where $\alpha \in (0, 1]$ is the learning rate. The quantity inside the square brackets is called the *temporal-difference error*.

For the snake problem, since there are three possible actions, the expected value of the next state can be written explicitly as:

$$\begin{aligned} \sum_{a' \in A} \pi_\varepsilon(a' | S_{t+1}) Q(S_{t+1}, a') &= (1 - \varepsilon) \max \{Q(S_{t+1}, \text{straight}), Q(S_{t+1}, \text{left}), Q(S_{t+1}, \text{right})\} \\ &\quad + \frac{\varepsilon}{3} [Q(S_{t+1}, \text{straight}) + Q(S_{t+1}, \text{left}) + Q(S_{t+1}, \text{right})] . \end{aligned}$$

Therefore, the complete learning rule for the snake problem is:

$$\begin{aligned} Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \left((1 - \varepsilon) \max \{Q(S_{t+1}, \text{straight}), Q(S_{t+1}, \text{left}), Q(S_{t+1}, \text{right})\} \right. \right. \\ \left. \left. + \frac{\varepsilon}{3} [Q(S_{t+1}, \text{straight}) + Q(S_{t+1}, \text{left}) + Q(S_{t+1}, \text{right})] \right) - Q(S_t, A_t) \right] . \end{aligned}$$

If S_{t+1} is a terminal state, meaning that the snake has hit a danger, the future value is zero. The update then becomes:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [-10 - Q(S_t, A_t)] .$$

Similarly, when the snake reaches the food, the immediate reward is +10, while when it reaches an empty cell the immediate reward is -0.1. By repeating this update over many episodes, actions leading to food acquire higher values, while actions leading to collisions acquire lower values.

During training, ε can be gradually reduced so that the agent explores frequently at the beginning and increasingly exploits the learned values afterward. Once training is complete, we set $\varepsilon = 0$ and select:

$$A_t = \arg \max_{a \in A} Q(S_t, a) .$$

15.7 Transformers

This section would require a bit more of structure than the others. Since transformers are mostly used for text processing, we will only refer to those examples. We will introduce the concept of *embedding*, we will discuss the

Tokenization and Embedding

Given a language, each word, or sentence can be divided into "atomic" portions that are called *tokens*. The tool that decomposes the sentence into tokens is called **tokenizer**, and it's crucial that it applies the best tokenization procedure possible. These tokens could be for instance the vocabulary, so that the sentence *"The dude sleeps comfortably."* would contain 5 tokens: *"The"*, *"dude"*, *"sleeps"*, *"comfortably"* and *"."*. However, this is hardly the case when dealing with real tokenizers, since they often split words into smaller subword units for semantical reasons. In this case for instance, it probably would have looked like: *"The"*, *"dude"*, *"sleep"*, *"s"*, *"comfort"*, *"ably"* and *"."*. The idea behind tokenization of audio or images is the same, we just need to identify which are the fundamental "bricks" of the context we are in.

There are many tokenization algorithms to be used to train a tokenizer. They all need a good chunk of data to read, with as many different characters as possible to build a comprehensive vocabulary. The most common algorithm is called *Byte Pair Encoding* (BPE) and it is an iterative process of merging most frequent pair of tokens that appear adjacently. It starts from a fragmented dictionary (character level) and pairs it with the following token. According to how many times that token-pair appears, it can become a new token itself, according to some thresholds. Ideally, this will create a semantical vocabulary with enough data. Note that languages based on *ideograms* are easier to tokenize, since ideograms are already semantical token by themselves! The vocabulary size – meaning the number of different tokens – in a model ranges typically from 30k to 200k tokens.

Once the tokenizer is built, we need an **embedder**. The embedder is a model that takes tokens and produces real N -dimensional vectors that can be mixed and compared together. Each token is identified by a unique vector in the embedding space, usually with a dimension $N < 1000$. This creates an *embedding matrix* with size $E_M \in \mathbb{R}^{N \times V_t}$, where V_t is the vocabulary size. The initial vectors are usually assigned randomly, then they are update jointly with the transformer's weights. In practice, the embedding matrix is part of the model parameters.

15.8 Model Performances Evaluation

Once we developed a model – imaging, forecasts, classifiers, etc. – we need to find a way to monitor "how good it is" and if its performances degrades over time. In order to do that, we need some metrics and good practices that give us hints on the state of the model. However, according to the setup we are working with, we might not be able to use all the evaluation techniques in every situation. For instance, if we are using an unsupervised model usually it's because we cannot know the ground truth – ex. clustering procedures – so we are not able to evaluate the confusion matrix.

Data Splitting

Here is a list of the most used data splitting techniques used in order to setup the datasets for training.

- **Train - Validation - Test** : Base idea for every ML applications based on supervised learning. Starting from a pool of data, we divide it in *train*, to tune the parameters of the model, *validation*, to fine-tune the parameters once the model has been developed, and *test*, to evaluate the real-world performance of the model.

- **K-Fold Cross-validation** : This is a statistical approach still based on train-test split, but it's aimed to give a more robust estimate of the performance of the model. We divide the full dataset in K parts and we train the model on $K - 1$ folds, and we test it on the remaining fold. We repeat this training process K times and we average the results.
- **Stratified Splitting** : Split the dataset into train-validation-test parts that preserve the class ratios. This is very important for unbalanced datasets, like anomaly detection, where you can have millions of sane data and just few anomalies.
- **Out of Distribution (OOD) Validation** : We remove one group, class, or family of records from the dataset, and we perform the normal training procedure. At the end, we test the model on the excluded data, so that we understand how well it behaves on heterogeneous records.

Supervised Metrics

Supervised learning consists in training and testing the model knowing the ground truth of our data. This grants us the ability to evaluate quantitatively the performances of the model, since we understand if its predictions are correct, wrong and how much they do deviate from the correct ones.

- **Accuracy** :

$$\frac{\text{Correct predictions}}{\text{Total predictions}}$$

- **Balanced Accuracy** :

$$\frac{1}{K} \sum_{k=1}^K \frac{\sum_i w_i \mathbf{1}(y_i = k, \hat{y}_i = k)}{\sum_i w_i \mathbf{1}(y_i = k)} = \frac{1}{K} \sum_{k=1}^K \text{Recall}_k$$

where w_i are the class sample weights (usually how many records of that class, but can be a custom importance), y_i is the real label, $\hat{y}_i$ is the predicted label, k is the current considered label and K is the number of labels – classes – in total. The function $\mathbf{1}(x)$ is the indicator function, and simply tells if the element given is part of that class or not.

- **Precision** : Given a class, the precision is

$$\frac{\text{Correct predictions in class}}{\text{Total predictions in class}}$$

which is basically the *accuracy* of a specific class.

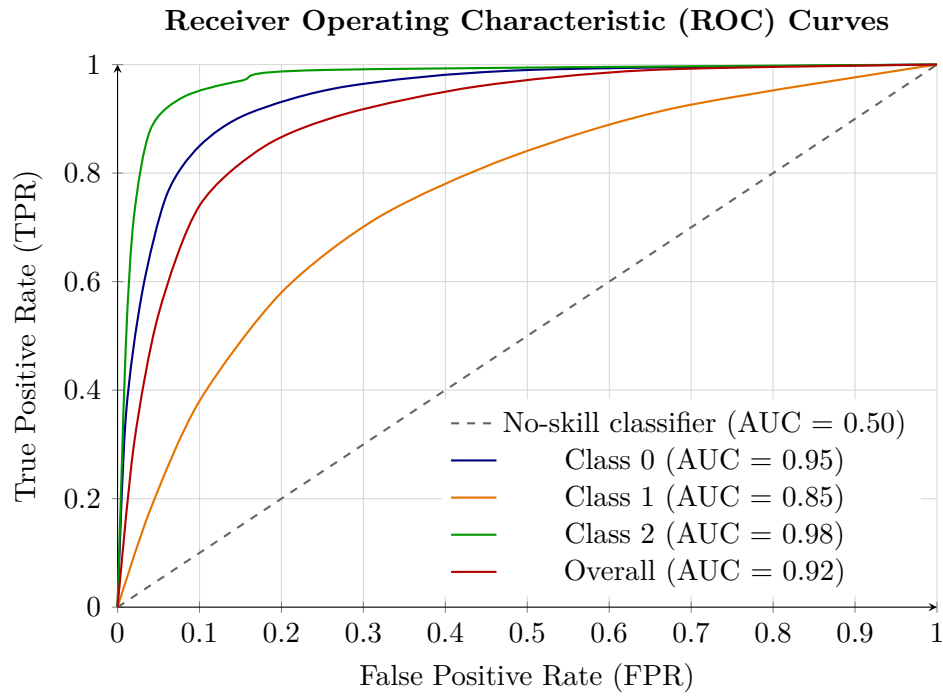
- **Recall** : Given a class, the recall is

$$\frac{\text{Correct predictions in class}}{\text{Real elements in class}}$$

- **F1 Score** : F1 is mostly used for imbalanced data since it weights in both precision and recall

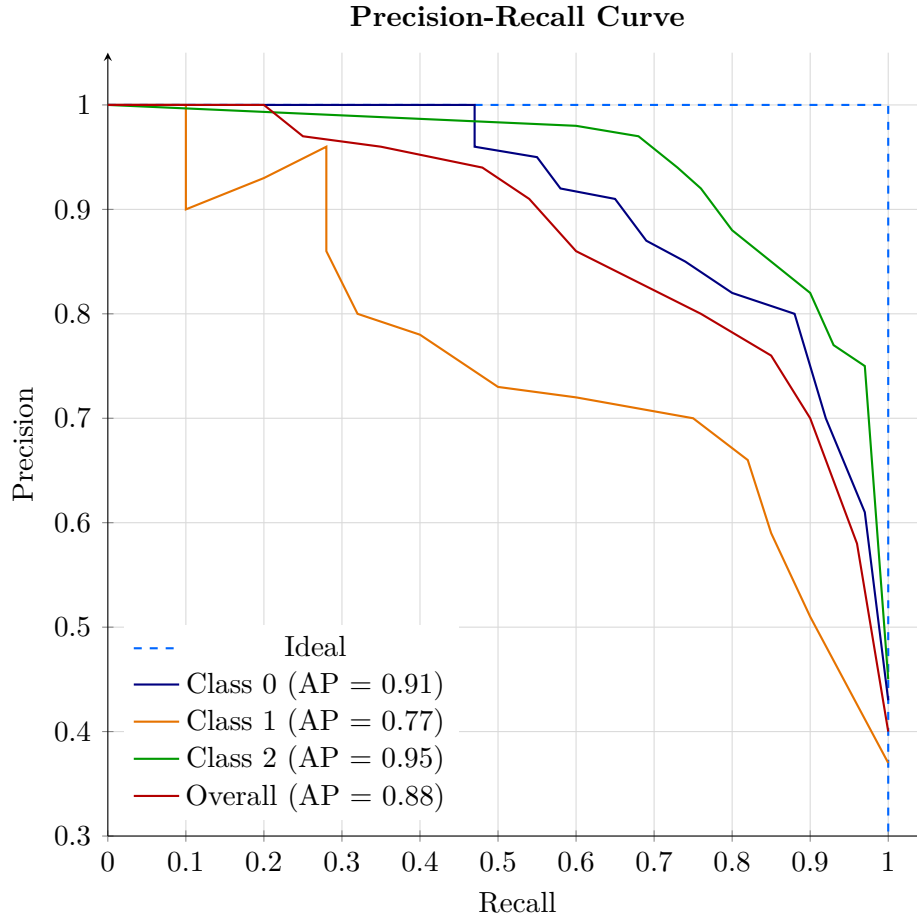
$$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **ROC - AUC** : Once a model predicts an object's label, or class, it typically assigns a probability score to that prediction. For a given class, we can define a threshold T : predictions with a probability greater than T are classified as belonging to that class, while those with probabilities below T labeled as not belonging to the class. Using this idea, we can plot the *ROC* Curve of the model:



The idea of "AUC" (Area Under the Curve) is just to give a quantitative summary of the ROC curve. Usually we plot more parameters configurations to see which model has performed best.

- **PR Curve** : The PR (Precision-Recall) Curve shares the same underling concept of the ROC one and the same evaluation technique, we only change the quantities that are plotted. The PR curve is better than the ROC if we value more the class prediction quality instead of the overall predictions, which is crucial for imbalanced data.



- **Mean Squared Error (MSE)** : It is simply the variance of the prediction

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 .$$

It's particularly useful since it's fast to calculate, and it gives a straightforward interpretation of how much the model nails the prediction overall. It is usually also evaluated the **Root Mean Squared Error (RMSE)**, which is simply $\sqrt{\text{MSE}}$, mostly because it has the same unit of measure of the predicted variable and do not penalize outliers errors as heavily as the MSE.

- **Mean Absolute Percentage Error (MAPE)** : This is a good indicator of model performances in terms of percentage of the real values

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \left| \frac{\hat{y}_i - y_i}{y_i} \right| .$$

However, this is extremely unstable for values close to zero.

- **Weighted Mean Absolute Percentage Error (WMAPE)** In order to avoid the instability of MAPE, we can weight each value by "weights" w_i that usually are the "actuals" (ex. total sales, volumes, etc..)

$$\text{WMAPE} = \frac{1}{\sum_{i=1}^N w_i} \sum_{i=1}^N w_i \left| \frac{\hat{y}_i - y_i}{y_i} \right| = \frac{\sum_{i=1}^N |y_i| \left| \frac{\hat{y}_i - y_i}{y_i} \right|}{\sum_{i=1}^N |y_i|} = \frac{\sum_{i=1}^N |\hat{y}_i - y_i|}{\sum_{i=1}^N |y_i|} .$$

This structure removes the issues around zero and it's particularly interesting from a business perspective since it takes into consideration the single importance of the quantities considered.

- **Mean Absolute Scaled Error (MASE)** : This is a time series metric and it's used to compare a forecast model with the *naive* forecast.

$$\text{MASE} = \frac{\frac{1}{N} \sum_{i \in \text{test}} |\hat{y}_i - y_i|}{\frac{1}{T-1} \sum_{t \in \text{training}} |y_t - y_{t-1}|}$$

It's important to notice the difference between numerator and denominator, where naive is evaluated on the training set, while the model on the testing set.

- **Cross-Entropy Loss** : It is a measure of goodness of prediction, but it also values the confidence that the model has for that prediction. The general definition is the average value:

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log p_{i,k} = -\frac{1}{N} \sum_{i=1}^N \log p_i$$

where N is the number of samples, K is the number of classes, $p_{i,k}$ is the probability of the element i to be in the class k and $y_{i,k}$ is if the element i belongs to the class k ; for this reason, we can rewrite it in a simpler way as the latter. It is an average because $y_{i,k}$ is the true probability to be in a class (which is 1 or 0), and $-\log p_i$ is the random variable we are averaging. Cross-Entropy loss is widely used in ML applications as gold standard loss function thanks to the significant consideration of the probability associated to the prediction.

16 Relativity

17 Quantum Mechanics

Review of the fundamental concepts and result of quantum mechanics. Quantum mechanics is based on wave functions and operators, so we will introduce the main mathematical tools to describe them and we will follow by defining the physics that uses them.

17.1 Hilber Space

We define Hilbert Space H a space with the following properties:

1. H is a vector space
2. H has inner product $\langle f, g \rangle$
3. Norm $\|f\| = \sqrt{\langle f, f \rangle}$
4. H is complete, so every Cauchy sequence converges in the space H .

In practice, we will always consider $H \sim L^2(a, b)$, where $L^2(a, b) = \{f : \int_a^b |f(x)|^2 dx < +\infty\}$ such that:

1. $L^2(a, b)$ is a vector space
2. Inner product $\langle f, g \rangle = \int_a^b \overline{f(x)} g(x) dx$
3. Norm $\|f\| = \langle f, f \rangle = \int_a^b |f(x)|^2 dx$
4. Complete with respect to the norm $\|f\|_2 = \sqrt{\int_a^b |f(x)|^2 dx}$.

This is useful for wave functions, since in quantum mechanics the squared wave function is a probability density function, so the wave functions live in an Hilbert space.

17.2 Nomenclature

The concepts expressed in quantum mechanics are mostly the same of linear algebra. However, since the elements of the vector space are usually functions (wave functions), they live in an infinite dimensional space. Considering Fourier decomposition, we know that we can express any function as an infinite sum of waves:

$$f(x) = \sum_{n=0}^{+\infty} F_n e^{inx} .$$

Here n is an integer that, in some sense, limits the number of allowed wavelengths according to their spacing. In the limit of the spacing going to zero, the same function can be expressed as an integral:

$$f(x) = \int_{-\infty}^{+\infty} F(\nu) e^{i\nu x} d\nu ,$$

with $\nu \in \mathbb{R}$ being the wave frequencies that are now continuous. As we will see, these two cases represents in quantum mechanics a confined observable that becomes discretized due to the dependency on the wavelength (like the energy of a particle), and an observable that is not affected by the quantization (like the position of a particle) respectively.

With this being said, the generators of the vector space can be either a numberable set of basis vectors (still infinite, but countable), or an uncountable set of functions that span the space via integrals. For this reason, a new nomenclature is used to pay proper attention to this fact, and to simplify the representation of some recurrent calculations.

- **column vector** : $\vec{v} = \begin{pmatrix} v_0 \\ \vdots \\ v_N \end{pmatrix} \xrightarrow{qm} |v\rangle$
- **row conjugate vector** : $\vec{v}^\top = (v_0^*, \dots, v_N^*) \xrightarrow{qm} \langle v|$
- **inner product** : $\langle v, u \rangle = \sum_{i=0}^N v_i^* u_i \xrightarrow{qm} \langle v|u \rangle = \int_a^b \overline{v(x)} u(x) dx$
- **matrix** : $A \xrightarrow{qm} \hat{A}$. Matrices are also called *operators*, since they are usually applied to functions.
- **Matrix-Vector multiplication** $\vec{u} = A\vec{v} \xrightarrow{qm} |u\rangle = \hat{A}|v\rangle$
- **Quadratic form** $\vec{u}^\top A \vec{v} \xrightarrow{qm} \langle u| \hat{A} |v \rangle$. The result is a scalar.

Since L^2 is a vector space, we can find an orthonormal basis $\{f_n(x)\}$ such that:

$$f(x) = \sum_{n=0}^{+\infty} F_n f_n(x) \quad (13)$$

whose coefficients F_n can be found exploiting the orthonormality:

$$\begin{aligned} \langle f_n(x) | f_m(x) \rangle &= \delta_{mn} \\ \Rightarrow F_n &= \langle f_n(x) | f(x) \rangle \quad (\text{Fourier method}) \end{aligned}$$

where we recall $f_n(x), f(x)$ are vectors in Hilbert space.

17.3 State Vectors, Basis and Descriptions

Once $L^2(a, b)$ and Hilbert spaces are defined, we ask what a vector in such space is. We call the *normalized* vector $|\Psi(t)\rangle \in H$ a **quantum state**, whose components depend on a variable t – usually the time. All the information about the physics of the system is embedded inside the quantum state. The reason why we want it to be normalized is related to **Born's rule** that we will see later.

Now, what is a quantum state $|\Psi\rangle$ exactly? It is an abstract object that lives in Hilbert space and represents a generic configuration of the particle. We need some basis of the space that has physical properties so that it can describe the state with concrete implications (energy, position, momentum, ...).

In particular, once we fix a vector basis $\{|q_i\rangle\}$ we are able to rewrite any possible state Ψ as a linear combination of that base – or superposition in this context, since we are dealing with probability waves.

The idea is simple: find a basis that represents something physical, then write the state of the particle according to that physical quantity. This is possible because we are in L^2 and so our states can be seen as wave functions.

Now we want to define this basis $\{|q_i\rangle\}$. Since we want real quantities, we know that **Hermitian** operators have real eigenvalues. This is useful, because now we can define the Hermitian operator $\hat{Q}$ such that:

$$\hat{Q} |q\rangle = q |q\rangle \quad (14)$$

where we ask that if a system is in a state $|q\rangle$, then a measurement of the physical quantity Q will yield the value q with probability 1. We must be smart to define $\hat{Q}$ such that it actually represents physical quantities and that they would eventually recover the classical counterpart in the classical limit.

If we manage to define $\hat{Q}$, then to get $\{|q_i\rangle\}$, we can write any state in Hilbert space as:

$$|\Psi\rangle = \sum_{n=0}^{+\infty} c_n |q_n\rangle \quad (15)$$

Here we have to pause for a second to highlight a crucial point about Hilbert space and basis. A quantum state $|\Psi\rangle$ can be given by a linear combination of *numerable* vectors $\{|q_i\rangle\}$. However, since we are in L^2 , we can also rewrite any vector $|\Psi\rangle$ as a decomposition

of functions using continuous integrals, exactly as Fourier decomposition:

$$f(x) = \sum_{n=-\infty}^{+\infty} F_n e^{inx}$$

$$f(x) = \int_{-\infty}^{+\infty} F(l) e^{ilx} dl$$

Depending on the physical situation we are in, we might end up in a situation, in another, or even in both. Some quantities in quantum mechanics depend on the **wavelength** of the particle, like the momentum and the energy. Others do not, like the position of the particle. If we consider a particle in a box, so in a potential well, it will have a boundary condition $\Psi(0) = \Psi(L) = 0$. This condition means that only some wavelengths are permitted, thus only some energies. The position is also limited within 0 and L , but in a continuous way. This means that not all observables have basis that lives in the Hilbert space! This doesn't affect that much in terms of discussion, but it is a profound property of L^2 spaces. The set of eigenvectors of an operator generate the **spectrum** of the operator. We will refer to **discrete spectrum** when the basis is numerable, and **continuous spectrum** when our state must be expressed as an integral – i.e. when the quantity depends on the wavelength or not.

Now, as we do in physics, we want to predict what could be the outcome of a real measure of the quantity Q . For the discrete spectrum, we know that the state is given as a superposition of vector basis, each representing a possible outcome of the measure by definition. So it's reasonable to assume that the projection of the state $|\Psi\rangle$ on the basis vector $|q_n\rangle$ gives us the probability amplitude to measure the value q_n when measuring Q . From this assumption, and from the fact that the basis is *orthonormal*, we can obtain one of the most important rules of quantum mechanics: **Born's rule**, which quantifies the probability of measuring a specific value among the available ones given by the basis as:

$$P(Q = q_i) = |\langle q_i | \Psi \rangle|^2 = \frac{|c_i|^2}{|k|^2} = |c_i|^2 \quad (16)$$

with k being the length of the state vector, that we asked to be normalized exactly for this reason.

For the continuous spectrum, the reasoning is the same, but as we did for Fourier decomposition, at this point we need to work with a continuous function of coefficients $\{c_n\} \rightarrow \Psi(q)$ such that:

$$\Psi(q) = \langle q | \Psi \rangle$$

$$\rightarrow |\Psi\rangle = \int_a^b \Psi(q) |q\rangle dq$$

and so the probability to measure the quantity Q with value between a range of values $[q_0, q_1]$ is:

$$\rho(q) = |\Psi(q)|^2$$

$$\rightarrow P(q_0 \leq Q \leq q_1) = \int_{q_0}^{q_1} \rho(q) dq$$

where $\rho(q)$ is the probability density function to measure a physical quantity of value q . Here we also understand that the wave function valued for a certain basis q will produce

a scalar (or a complex number):

$$\psi : \mathcal{Q} \longrightarrow \mathbb{C}, \quad q \longmapsto \psi(q) = \langle q | \Psi \rangle.$$

This is exactly the same thing that happens in Fourier analysis: we can rewrite any function $f(x)$ as a superposition of different waves with different amplitudes, which in the limit of the distance between two consecutive wavelengths going to zero, it becomes an integral over all possible wavelengths. So we rewrite the function $f(x)$ in terms of a new continuous function $F(\nu)$, with ν being a variable in a continuous domain (**frequency domain**); in quantum mechanics we rewrite our state Ψ in terms of a new continuous function $\Psi(q)$, where q is a continuous variable defined in a specific physical space ($q = x \rightarrow$ **space domain**, $q = p \rightarrow$ **momentum domain**, ...).

Note that each wave function Ψ_p denoting the state of particle p lives in a Hilbert space H_p . If we consider two non-interacting particles x and y , the joint state will be described as:

$$|\Psi_{x,y}\rangle = |\Psi_x\rangle \otimes |\Psi_y\rangle$$

which lives in a new Hilbert space $H_x \otimes H_y$ and if we fix a basis q we do get, as we will see later:

$$\Psi_{x,y}(q_x, q_y) = \Psi_x(q_x) \cdot \Psi_y(q_y) .$$

It is required that the particles do not interact, so that the Hamiltonian is separable, which is rarely happening due to EM interactions. Also pay attention to the fact that we are doing tensor product, not cross products – otherwise we wouldn't be able to reproduce all the spaces!

So to summarize:

- A Hermitian operator $\hat{Q}$ is chosen to be associated to a real observable Q .
- According to the eigenvector basis we use to represent our quantum state $|\Psi\rangle$ we give a specific description of the state (position, momentum, energy, ...).
- The quantum state $|\Psi\rangle$ can be written as a linear combination of the eigenvectors $|q_i\rangle$ of $\hat{Q}$. Measurements of Q yield the eigenvalues associated with these eigenvectors, with some probability to occur.
- Born's rule allows to find the probability of measuring specific values of Q .
- According to the type of observable we are considering (quantized or not), Born's rule will get a *probability* ($P(Q = q_i)$) or a *probability density* ($\rho(q)$). Continuous spectra allows continuous quantities, while discrete spectra allows only the eigenvalues of the eigenvectors associated to the quantized observable.

All of this is supported by experimental evidences, which is kinda incredible to think that this actually holds.

17.4 Define Observable Operators

The only thing left to do is how to define the operators representing physical quantities.

17.4.1 Commutator

We start by the assumption that we can describe any physical observable by finding its Hermitian operator $\hat{Q}$, whose eigenvalues are the possible results of a measurement. Then we need to introduce the concept of **commutator operator**:

$$[\hat{A}, \hat{B}] = \hat{A}\hat{B} - \hat{B}\hat{A}$$

which tells us if we can switch the order of application of two operators. Now, since we said:

$$\hat{Q} |q\rangle = q |q\rangle$$

if two operators commute, they will share the same orthonormal basis of vectors $\{|q_i\rangle\}$, in fact:

$$\begin{aligned} [\hat{A}, \hat{Q}] |q\rangle &= (\hat{Q}\hat{A} - \hat{A}\hat{Q}) |q\rangle = 0 \\ \Rightarrow \hat{Q}\hat{A} |q\rangle &= \hat{A}\hat{Q} |q\rangle = q\hat{A} |q\rangle \end{aligned}$$

which means that $\hat{A}$ must have the same eigenvectors of $\hat{Q}$ in order to commute. If two operators commute, we usually write the state of the system in terms of the eigenvalues of both operators for comodity:

$$|q, a\rangle : \{\hat{Q} |q, a\rangle = q |q, a\rangle, \hat{A} |q, a\rangle = a |q, a\rangle\}$$

since they share the same eigenvector basis. We said that an eigenvalue q is the certain result of a measurement of an observable Q when the system is exactly in the state $|q\rangle$. This means that for two commuting operators, we understand that we can measure both the observables Q and A with certainty if the system is in the state $|q, a\rangle$.

If the two operators do not commute, so if they have a non-zero commutator $[\hat{A}, \hat{B}] \neq 0$, then an important property arises: the **uncertainty principle**. This means that it doesn't exist a state $|\Psi\rangle$ in which we can measure at the same time the two observables Q and A with certainty, since they have different eigenvectors.

17.4.2 Generating Observables

The process of generating the "observable operator" for a specific physical quantity is non trivial and quite empirical. We usually need to start from physical relations in the observable space, then we manipulate them in order to extract a linear operator relationship, and with that we define the observable.

The energy operator is a good example to understand the process. We start with the one dimensional wave function:

$$\Psi(x, t) = e^{i(kx - \omega t)}$$

and we consider its time derivative:

$$\frac{\partial \Psi(x, t)}{\partial t} = -i\omega e^{i(kx - \omega t)} = -i\omega \Psi(x, t) .$$

We can use De Broglie relation $E = \hbar\omega$ to rewrite:

$$\frac{\partial \Psi(x, t)}{\partial t} = -i \frac{E}{\hbar} \Psi(x, t) ,$$

that can finally be written as:

$$E\Psi(x, t) = i\hbar \frac{\partial \Psi(x, t)}{\partial t} .$$

At this point, we notice that we are actually applying an "operator" to the wave function $\Psi(x, t)$ that extracts the *energy* of the function. We can rewrite it as:

$$E\Psi(x, t) = \hat{E}\Psi(x, t) , \quad \hat{E} = i\hbar \frac{\partial}{\partial t} .$$

In this way, we defined the **energy operator** for a system that can be described by a wave function $\Psi(x, t)$ (which are all systems in quantum mechanics).

Here is the list of all most notable observables operators:

- **Position Operator** : $\hat{q} = q$
- **Momentum Operator** : $\hat{p} = -i\hbar \nabla$
- **Angular Momentum Operator** : $\hat{L} = q \times -i\hbar \nabla$
- **Energy Operator** : $\hat{E} = i\hbar \frac{\partial}{\partial t}$

Now that we defined all the operators, we can actually prove **Heisenberg Uncertainty**

17.5 Schrödinger Equation

Schrödinger equation the exact equivalent of Newton's second law of Dynamics for quantum systems. It comes from empirical observations regarding both the wave-like probabilistic behavior of quantum particles, and de Broglie particle wavelength assumption – in addition to the already known photon's wavelength.

The idea is to define for a single particle a function $\Psi(x, t)$ that For a single non-relativistic particle of mass m , moving in a potential $V(\mathbf{r}, t)$, the time-dependent Schrödinger equation is given by:

$$i\hbar \frac{\partial \Psi}{\partial t}(x, t) = \left[-\frac{\hbar^2}{2m} \nabla^2 + V(x, t) \right] \Psi(x, t)$$

which if we consider a steady state – thus we remove the time dependency – we get:

$$-\frac{\hbar^2}{2m} \nabla^2 \psi(x) + V(x)\psi(x) = E\psi(x)$$

where we decomposed the wave function in two pieces $\Psi(x, t) = \psi(x)\phi(t)$.

Since we are assuming that the wave function is a probability density function, it should be normalized in L^2 space:

$$\int_{-\infty}^{+\infty} |\Psi(x, t)|^2 dx = 1$$

In Dirac notation we rewrite all the relations above:

$$i\hbar \frac{\partial}{\partial t} |\Psi(t)\rangle = \hat{H}(t) |\Psi(t)\rangle \longrightarrow \text{Schrödinger Equation} \quad (17)$$

$$\hat{H} = \left[-\frac{\hbar^2}{2m} \nabla^2 + V(x) \right] \longrightarrow \text{Hamiltonian Operator} \quad (18)$$

$$\langle \Psi(t) | \Psi(t) \rangle = 1 \longrightarrow \text{Normalization Condition} \quad (19)$$

Dirac notation is useful to express quantum states as elements of Hilbert space, without having to refer to a specific basis.

It's important to notice that given a system of two particles p_1 and p_2 the system is described by the equation (17). However, the system is not separable into single pieces as $|\Psi(t)\rangle = |\Psi_{p_1}(t)\rangle \otimes |\Psi_{p_2}(t)\rangle$, because of the interaction terms between the two particles in the Hamiltonian, expressed as potential: